

# GitHub Copilot



# Overview of GitHub Copilot



The GitHub Copilot Service is an **AI/GAI** powered code completion tool developed by GitHub in collaboration with OpenAI.

# What is Generative AI?

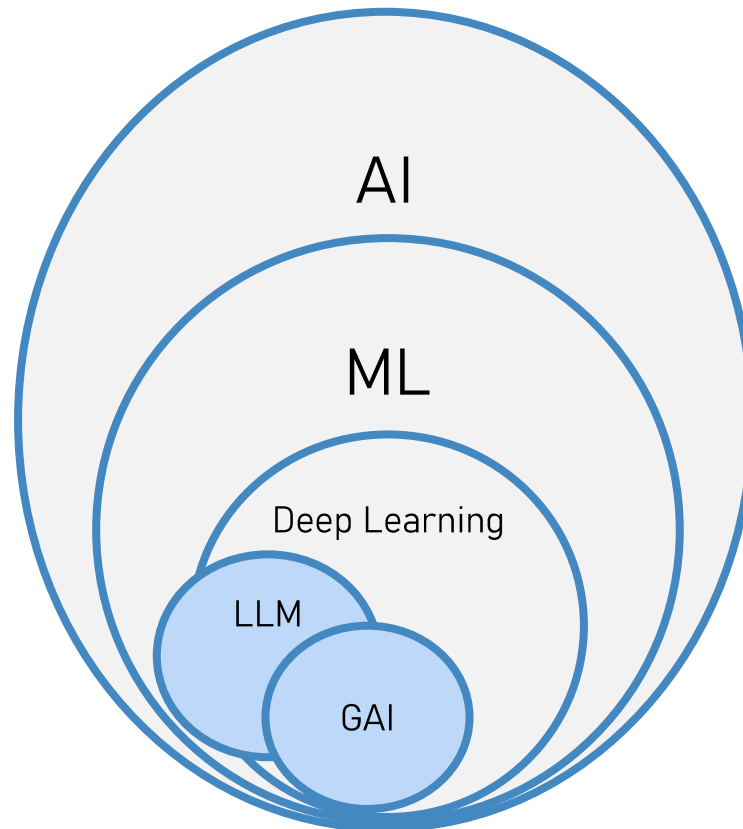


- Gen AI is a type of Artificial Intelligence that creates new content based on what it has learned from existing content.
- Generative AI was introduced in the 1960s in chatbots.
- In 2014 on introduction of GANs a type of machine learning algorithm that generative AI could create convincingly authentic images, videos and audio of real people.
- In 2017 Googles Brain team came up with Transformers as another ML algorithm with Deep Learning capabilities using LLM's.

# What is Generative AI?



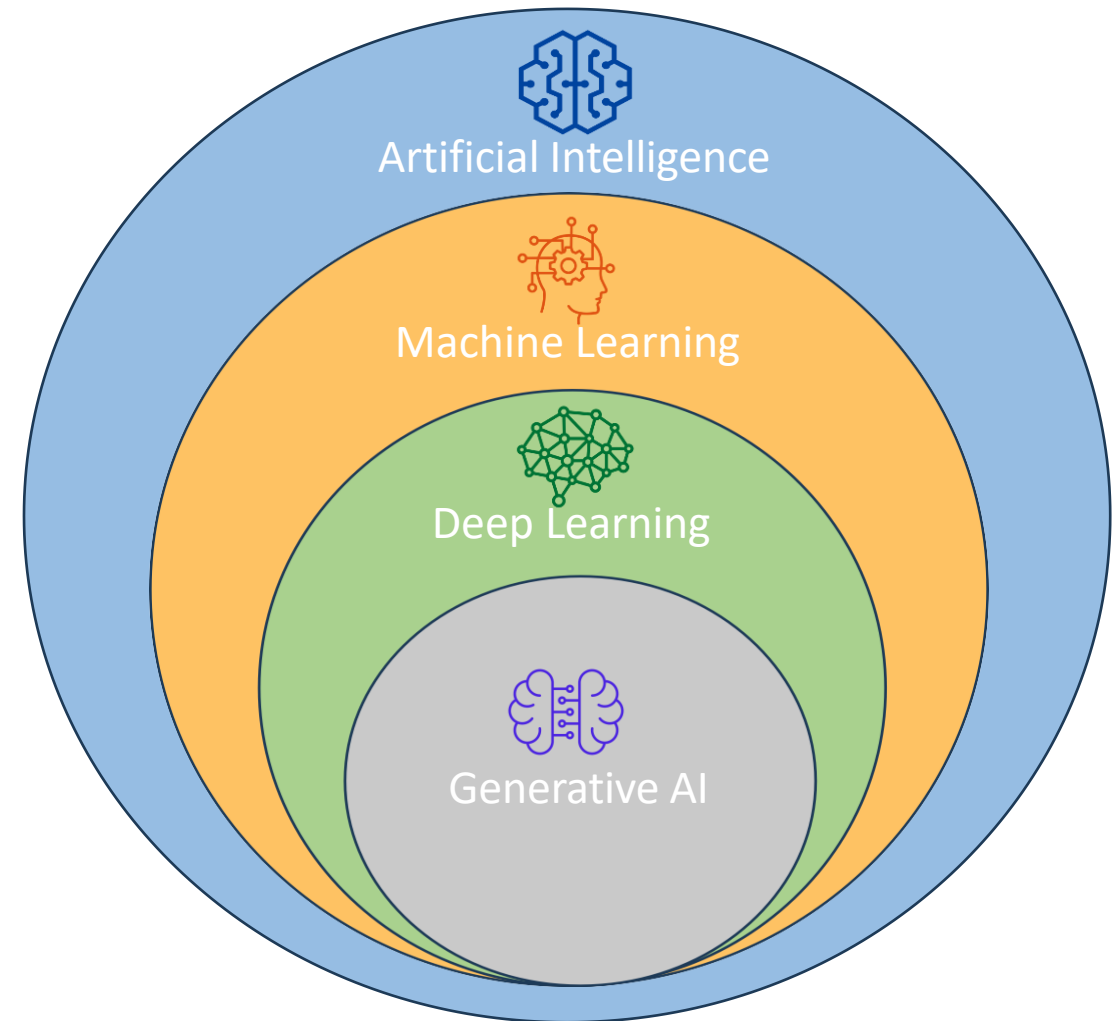
Generative AI is a type of artificial intelligence technology that can produce various types of content, including text, imagery, audio and synthetic data.



# What is Artificial Intelligence (AI)?



- AI is a broad field for the development of intelligent systems capable of performing tasks that typically require human intelligence:
  - Perception
  - Reasoning
  - Learning
  - Problem solving
  - Decision-making
  - Umbrella-term for various techniques

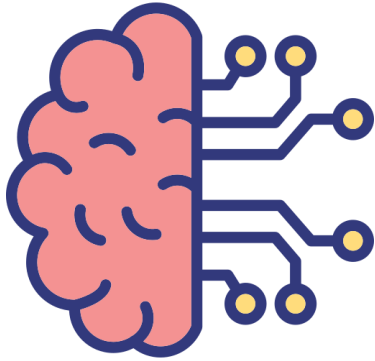


# What is Machine Learning?

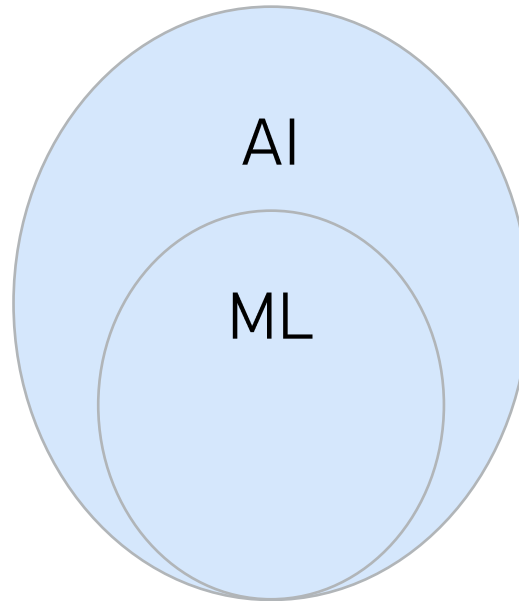


Machine learning is a branch of **artificial intelligence (AI)** and computer science which focuses on the use of **data** and **algorithms** to imitate the way that humans learn, gradually improving its accuracy.

# How AI and ML Related

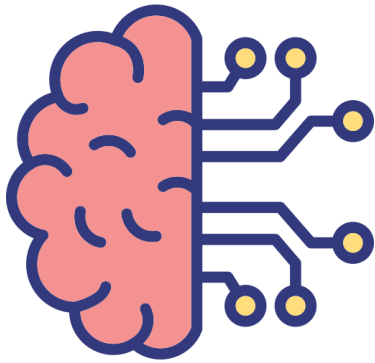


Artificial Intelligence  
is a discipline

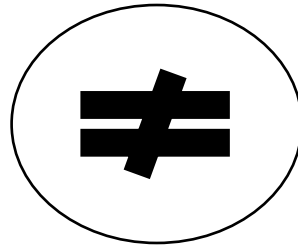


Machine Learning  
is a subfield

# AI $\neq$ ML



Artificial Intelligence



Machine Learning



## Ex: MYCIN Expert System

- System developed in 1970s to diagnose patients based on **reported symptoms and medical test results**
- Collection of over 500 rules
- Simple yes/no or textual questions
- It provides a list of culprit bacteria ranked from high to low based on the probability of diagnosis, the reason behind the diagnosis, and a potential dosage for the cure
- Never really used in production as personal computers didn't exist yet

|         |                                                            |
|---------|------------------------------------------------------------|
| RULE060 |                                                            |
| IF:     | THE IDENTITY OF THE ORGANISM IS BACTEROIDES                |
| THEN:   | I RECOMMEND THERAPY CHOSEN FROM AMONG THE FOLLOWING DRUGS: |
|         | 1 - CLINDAMYCIN (.99)                                      |
|         | 2 - CHLORAMPHENICOL (.99)                                  |
|         | 3 - ERYTHROMYCIN (.57)                                     |
|         | 4 - TETRACYCLINE (.28)                                     |
|         | 5 - CARBENICILLIN (.27)                                    |

|         |                                                                                                               |
|---------|---------------------------------------------------------------------------------------------------------------|
| RULE145 |                                                                                                               |
| IF:     | 1) THE THERAPY UNDER CONSIDERATION IS ONE OF: CEPHALOTHIN CLINDAMYCIN ERYTHROMYCIN LINCOMYCIN VANCOMYCIN, AND |
|         | 2) MENINGITIS IS AN INFECTIOUS DISEASE DIAGNOSIS FOR THE PATIENT                                              |
| THEN:   | IT IS DEFINITE (1) THE THERAPY UNDER CONSIDERATION IS NOT A POTENTIAL THERAPY FOR USE AGAINST THE ORGANISM    |

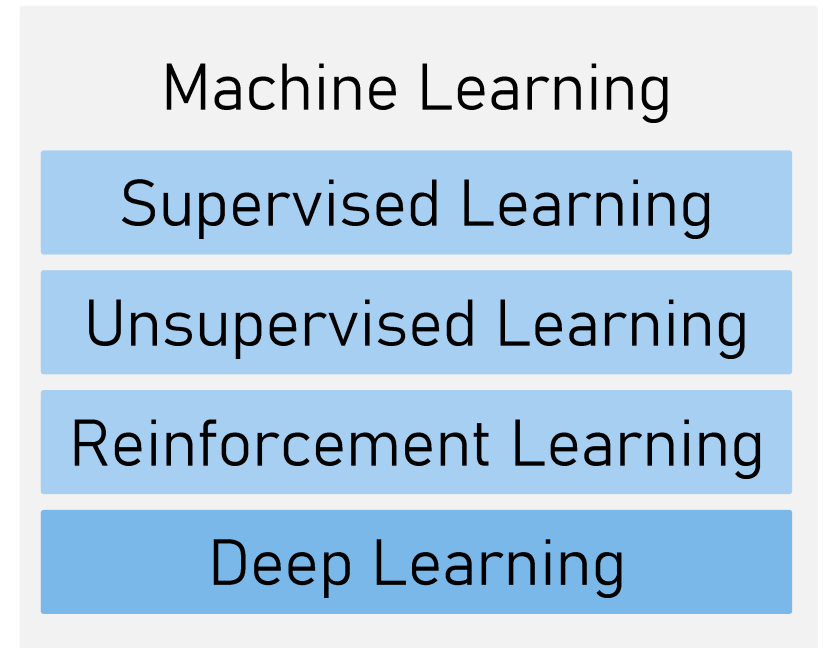
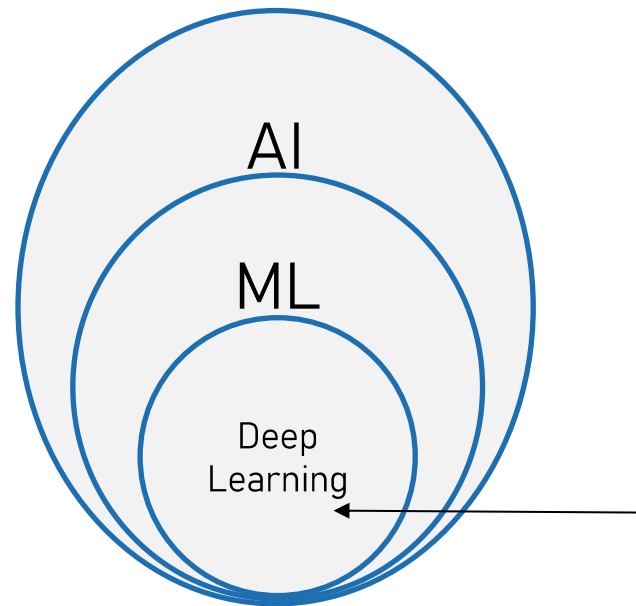
  

|         |                                                                                                 |
|---------|-------------------------------------------------------------------------------------------------|
| RULE037 |                                                                                                 |
| IF:     | 1) THE IDENTITY OF THE ORGANISM IS NOT KNOWN WITH CERTAINTY, AND                                |
|         | 2) THE STAIN OF THE ORGANISM IS GRAMNEG, AND                                                    |
|         | 3) THE MORPHOLOGY OF THE ORGANISM IS ROD, AND                                                   |
|         | 4) THE AEROBICITY OF THE ORGANISM IS AEROBIC                                                    |
| THEN:   | THERE IS STRONGLY SUGGESTIVE EVIDENCE (.8) THAT THE CLASS OF THE ORGANISM IS ENTEROBACTERIACEAE |

# Deep Learning



Deep learning is a subset of ML



# Deep Learning

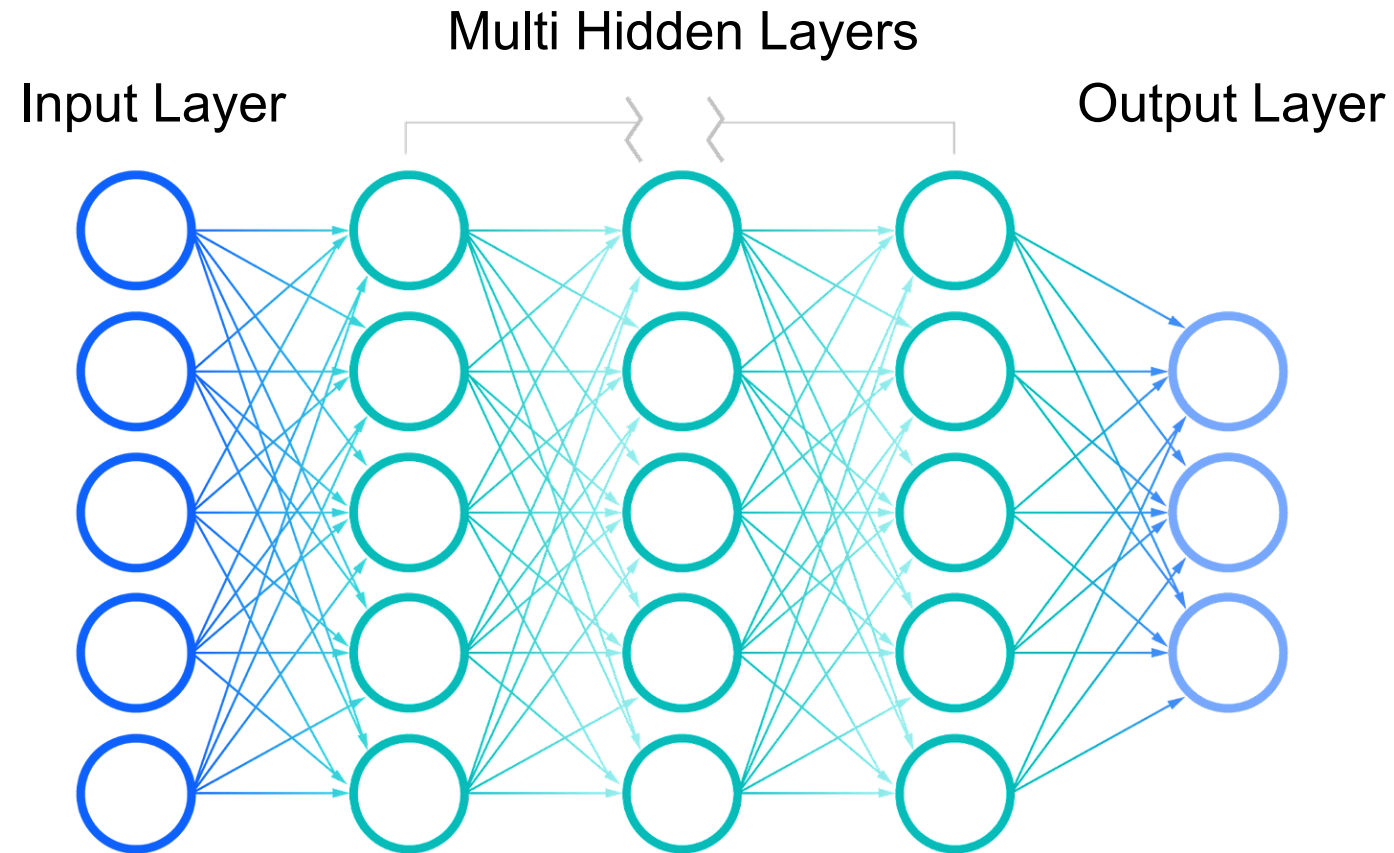


Deep learning uses Artificial Neural Networks - allowing them to process more complex patterns than traditional machine learning

- Uses neurons and synapses (like our brain) to train a model
  - Deep Learning because there's more than one layer of learning
    - Ex: Computer Vision – image classification, object detection, image segmentation
    - Ex: Natural Language Processing (NLP) – text classification, sentiment analysis, machine translation, language generation
- Large amount of input data Requires GPU (Graphical Processing Unit)



# Deep Neural Network



# Deep Learning Model Types



| Aspect                   | Discriminative Models                               | Generative Models                                           |
|--------------------------|-----------------------------------------------------|-------------------------------------------------------------|
| <b>Purpose</b>           | Used to classify or predict                         | Used to classify, predict, and generate data                |
| <b>Training Data</b>     | Typically trained on a dataset of labeled data      | Typically trained on a dataset of labeled data              |
| <b>Learning Approach</b> | Learns the relationship between features and labels | Learns the relationship between features and labels         |
| <b>Visualization</b>     | Defines a decision boundary between classes         | Can generate new samples based on learned data distribution |
| <b>Example Task</b>      | Classifying if an image is a cat or dog             | Generating new images of cats or dogs                       |

# Deep Learning Model Types



Discriminative  
Technique



Classify

Discriminative Technique  
(classify as a dog or a cat)



Discriminative  
Technique

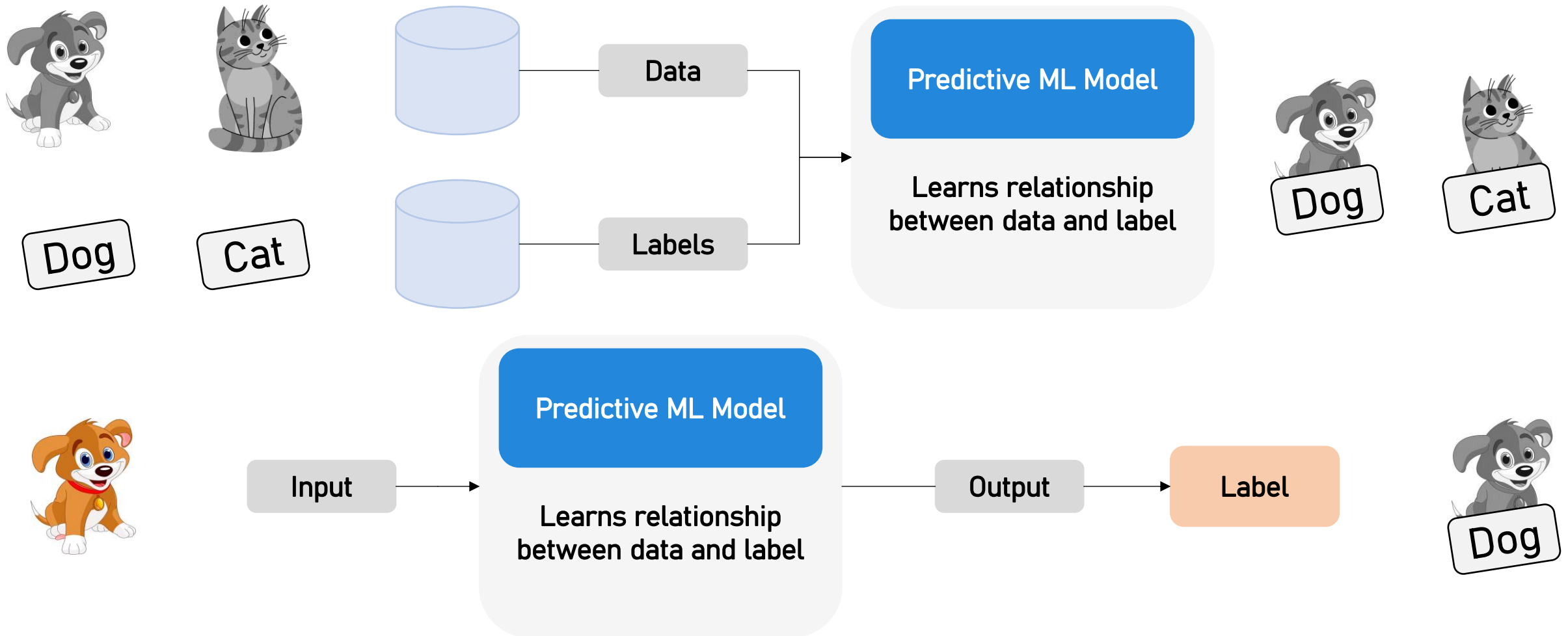


Classify

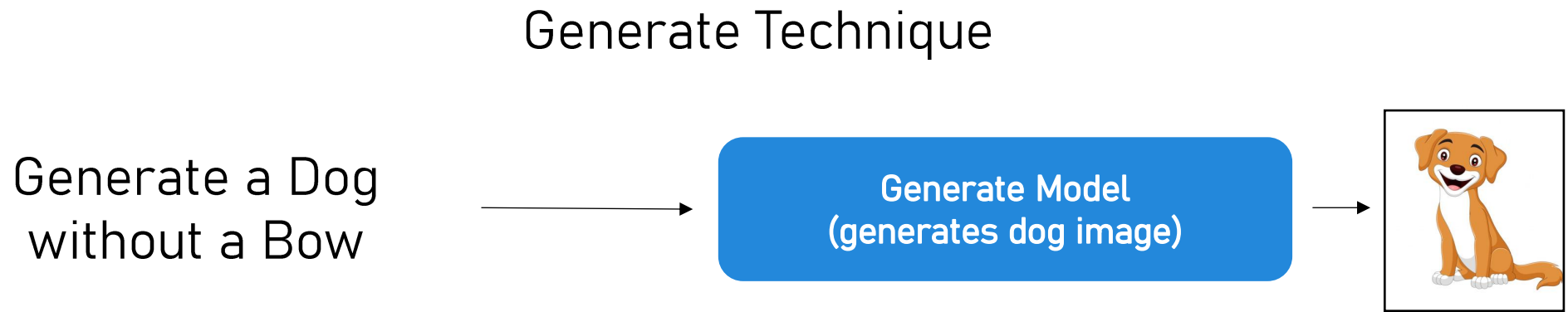
Discriminative Technique  
(classify as a dog or a cat)



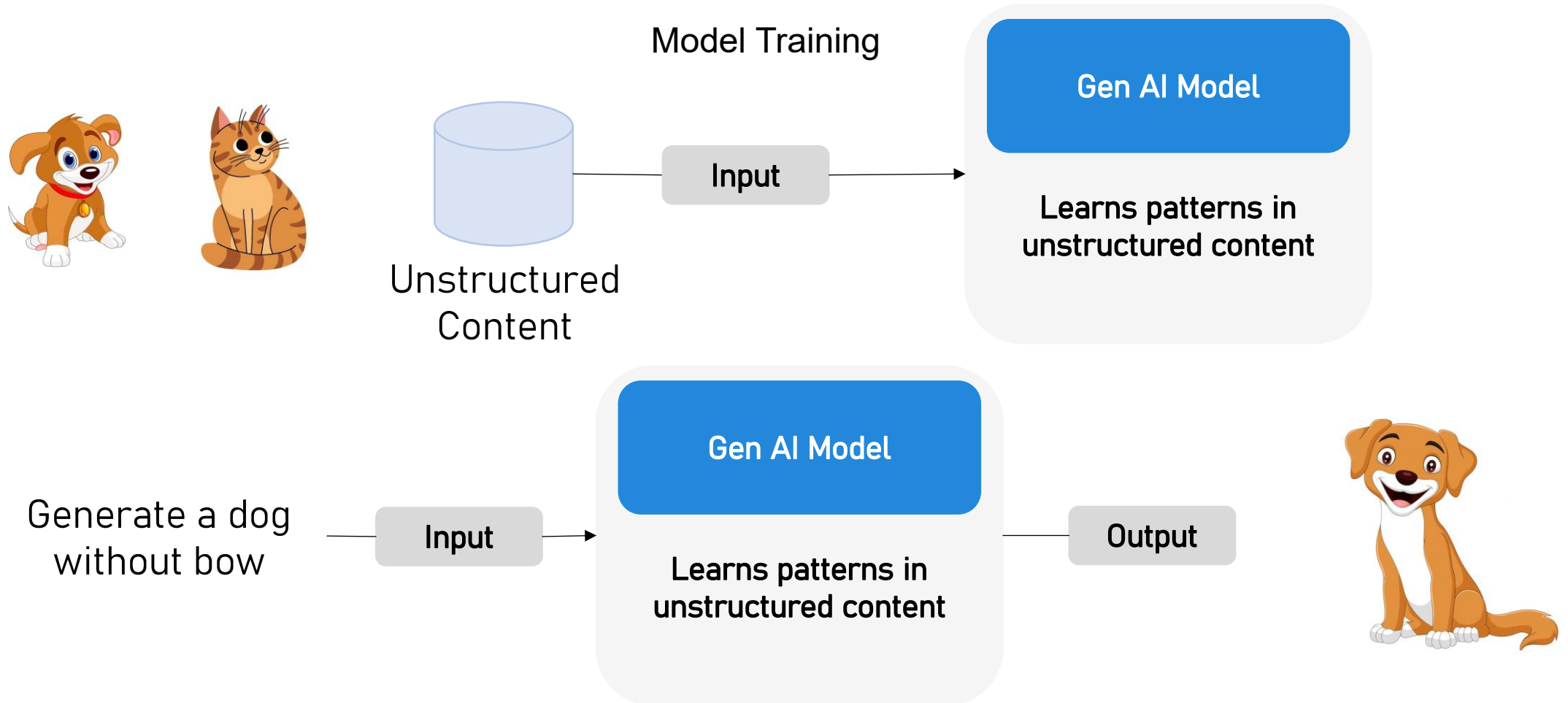
# Deep Learning Model Types



# Deep Learning Model Types

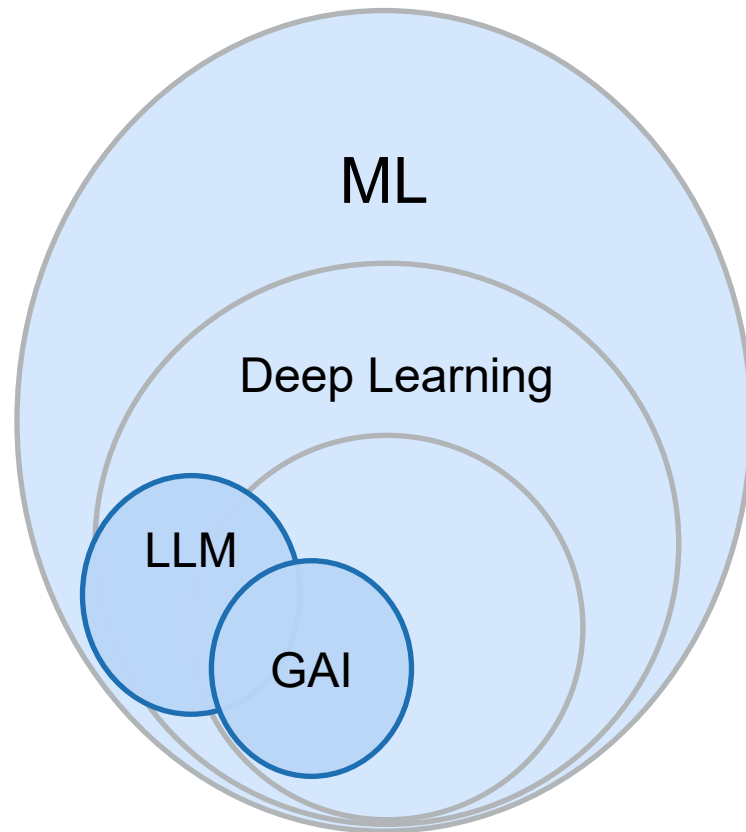


# Deep Learning Model Types





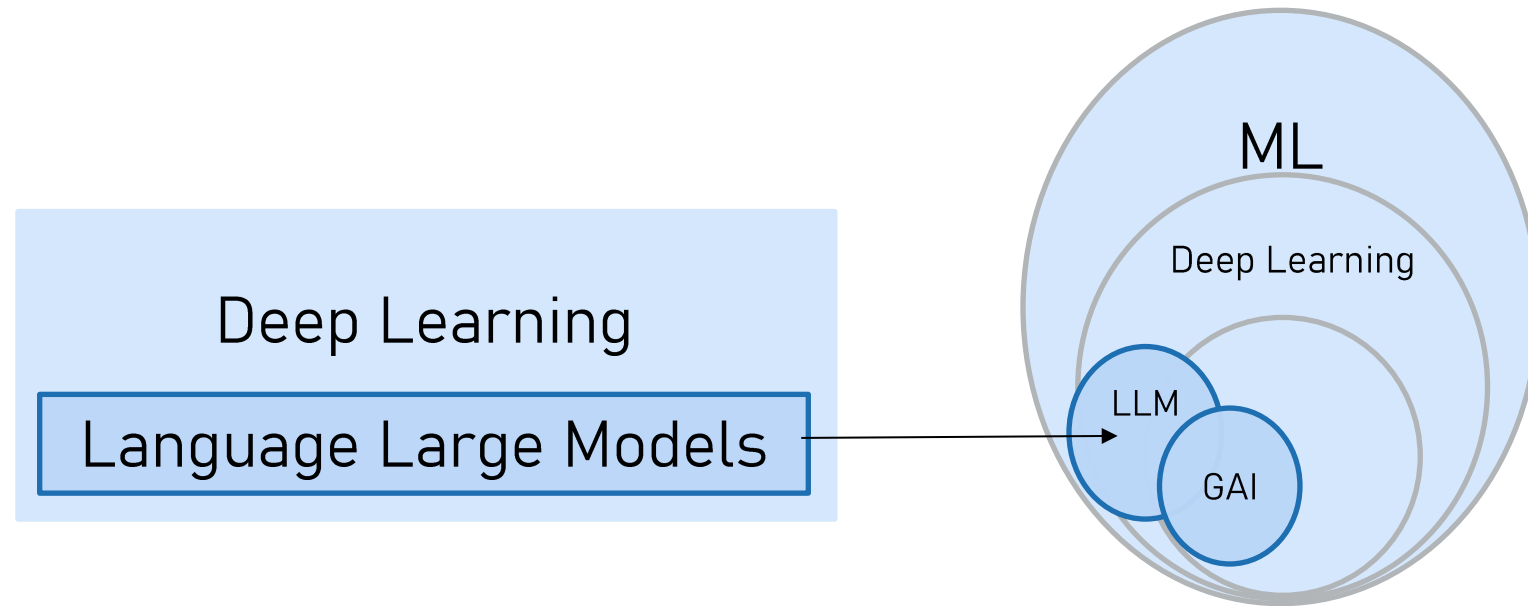
- Generative AI is a subset of Deep Learning which uses Neural Network



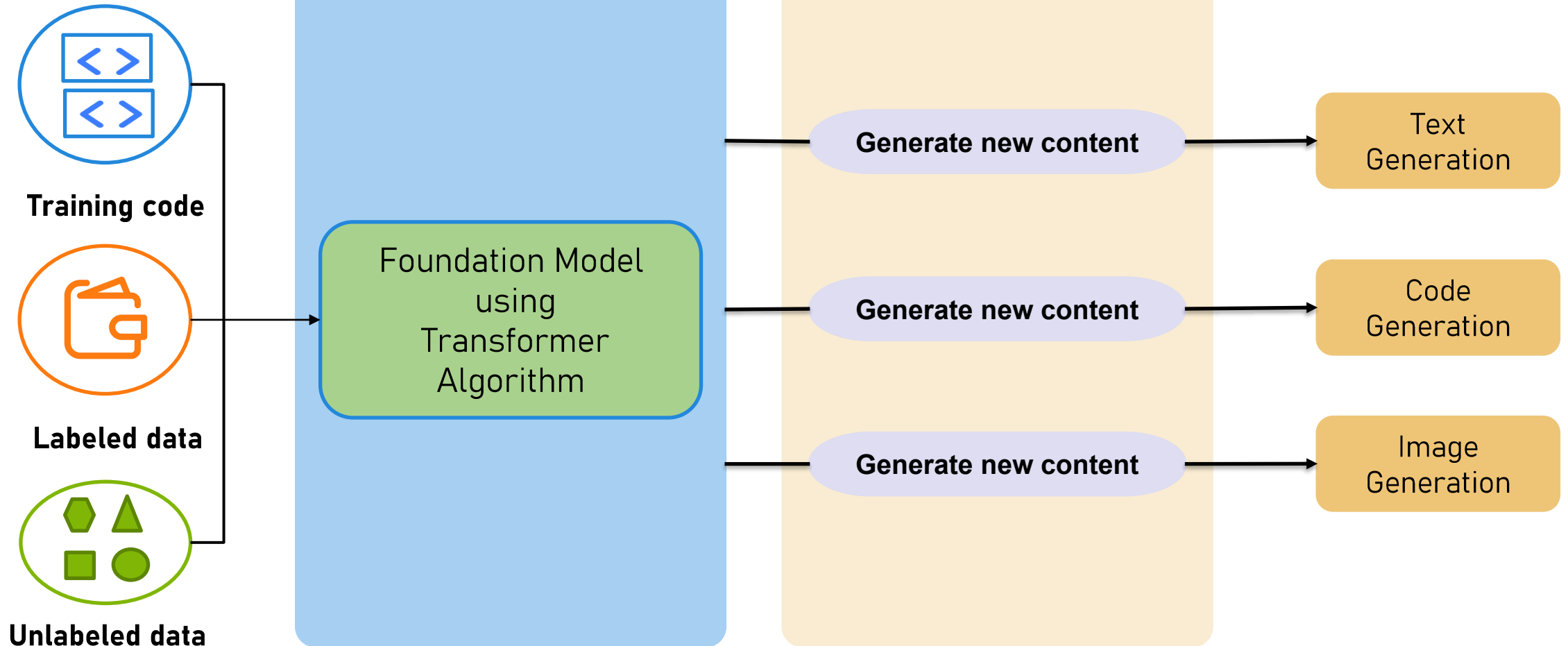
# Gen-AI Large Language Models



- Large Language Models (LLMs) are also a subset of Deep Learning



# Foundation LLM Models

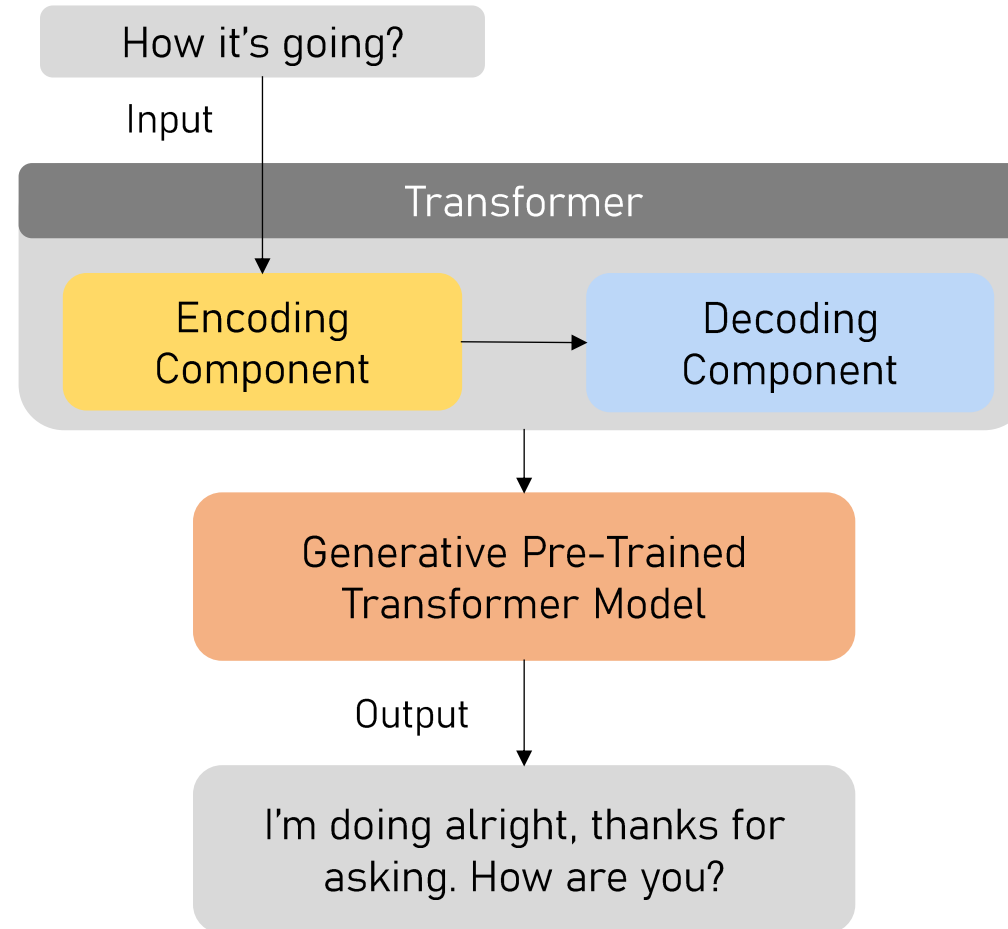


# How it Works



## Pre-Training:

- Large amount of Data
- Billions of parameters
- Unsupervised learning



# Transformers



- A transformer is a deep learning architecture developed by Google and based on the multi-head attention mechanism, proposed in a 2017 paper "Attention Is All You Need".[1] It has no recurrent units, and thus requires less training time than previous recurrent neural architectures, such as long short-term memory (LSTM)

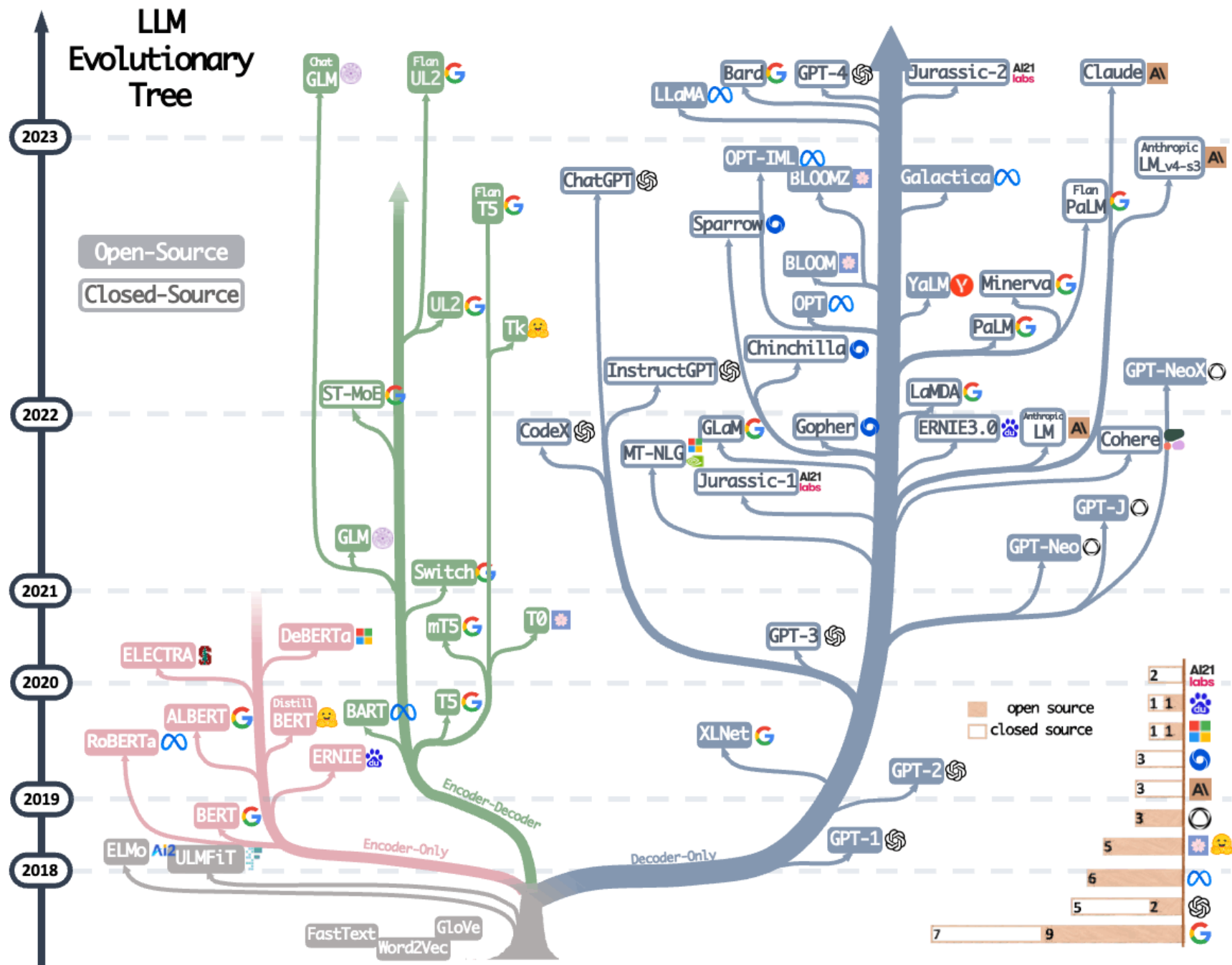
# Evolution of GAI [2017]

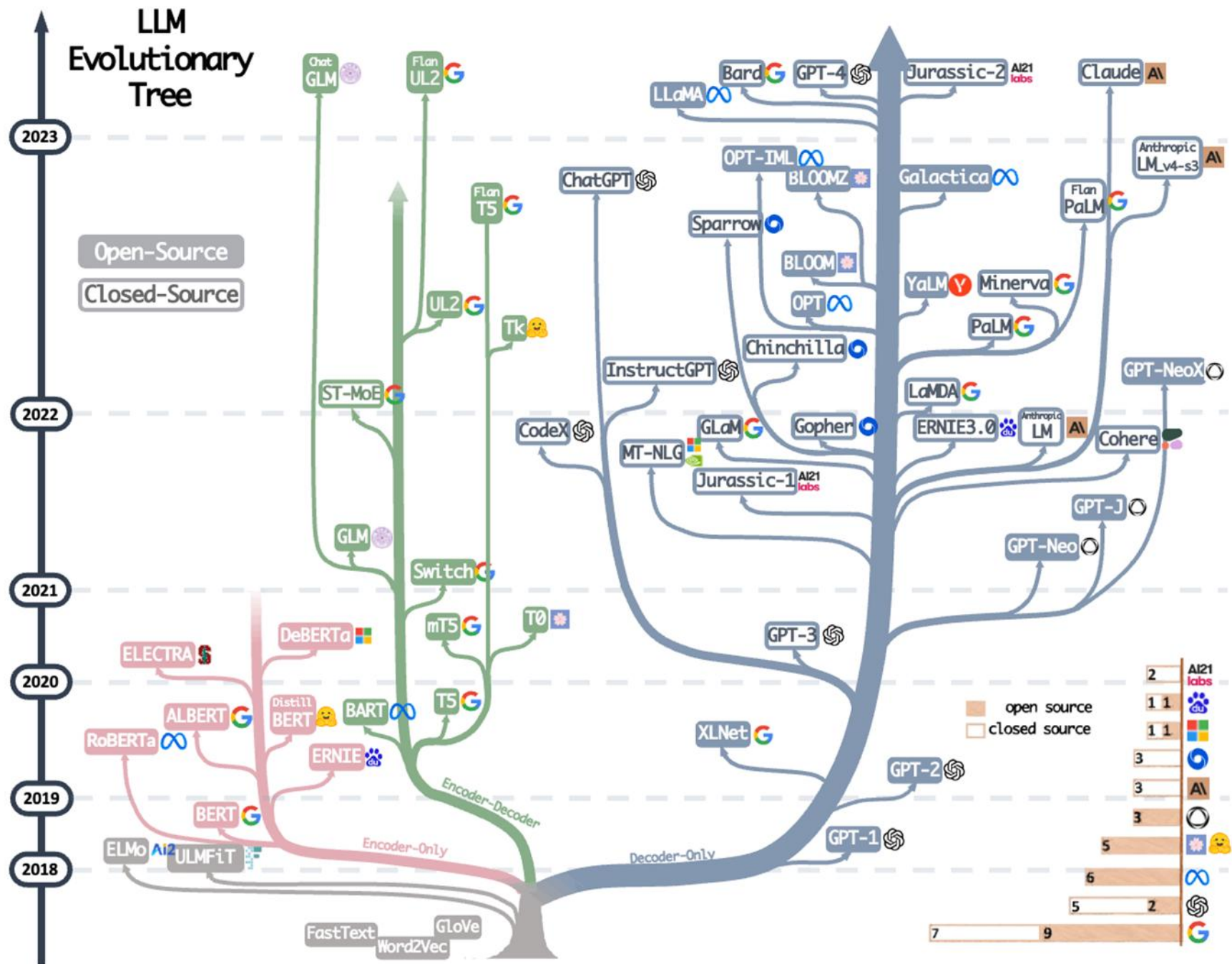


## Transformers



- Scale efficiently
- Parallel process
- Attention to input meaning





# Parameters



QUESTION ANSWERING

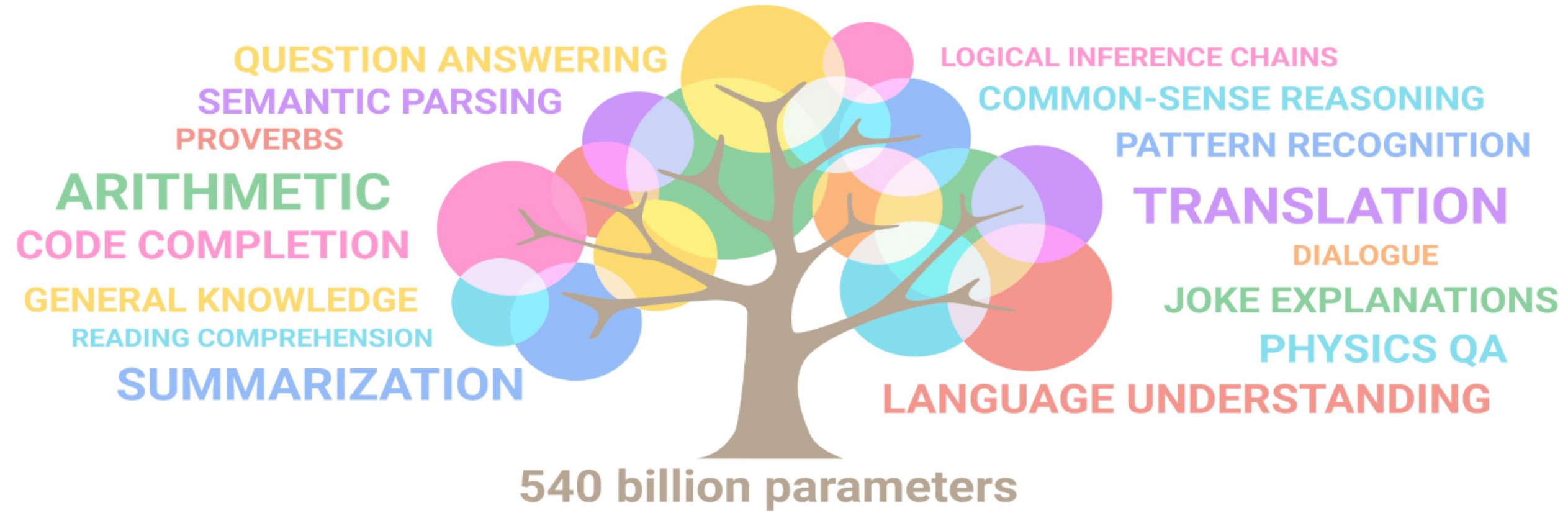
ARITHMETIC



LANGUAGE UNDERSTANDING

8 billion parameters

# Parameters



# Why LLM models are big?



## Llama 13B Model Card

- CCNet [67%]
- C4 [15%]
- GitHub [4.5%]
- Wikipedia [4.5%]
- Books [4.5%]
- ArXiv [2.5%]
- Stack
- Exchange[2%]
- 20 languages
- 2 Trillion Tokens



- Open source since 2007, available till June 2023
- 3 - 5 billion new pages added every month
- Every LLM training this corpus
- It contains 3.1 billion web pages or 370 TiB
- Page captures are from 44 million hosts or 35 million registered domains
- 82% raw tokens used in GPT-3

# What is Generative AI



- GenAI is a type of Artificial Intelligence that creates new content based on what it has learned from existing content.
- The process of learning from existing content is called training and results in the creation of a statistical model ( LLM ).
- When given a prompt, GenAI uses this statistical model ( LLM ) to predict what an expected response might be-and this generates new content.

# Generative Models



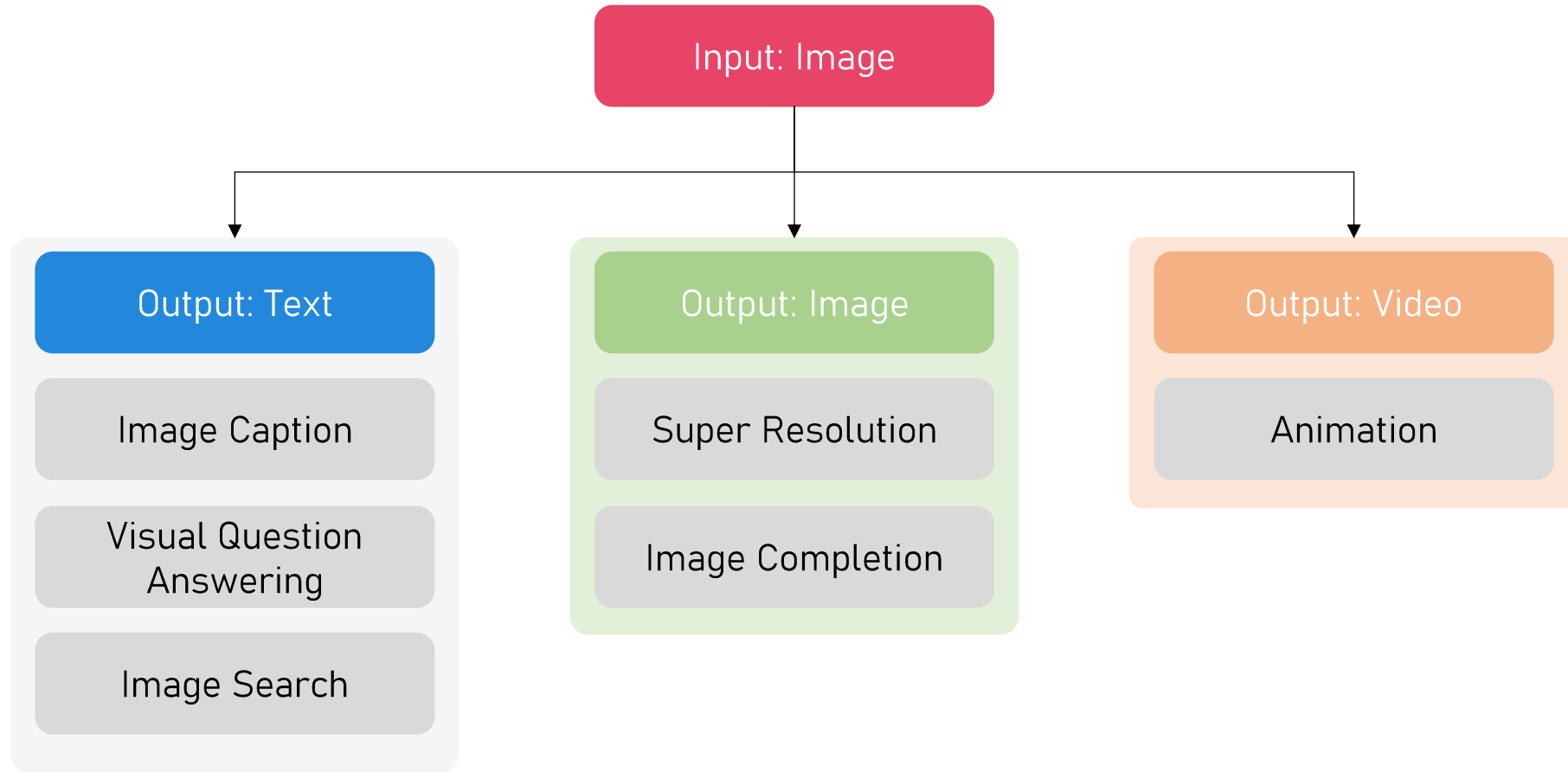
## Generative language models

- Generative language models learn about patterns in language through training data.
- Then, given some text, they predict what comes next.

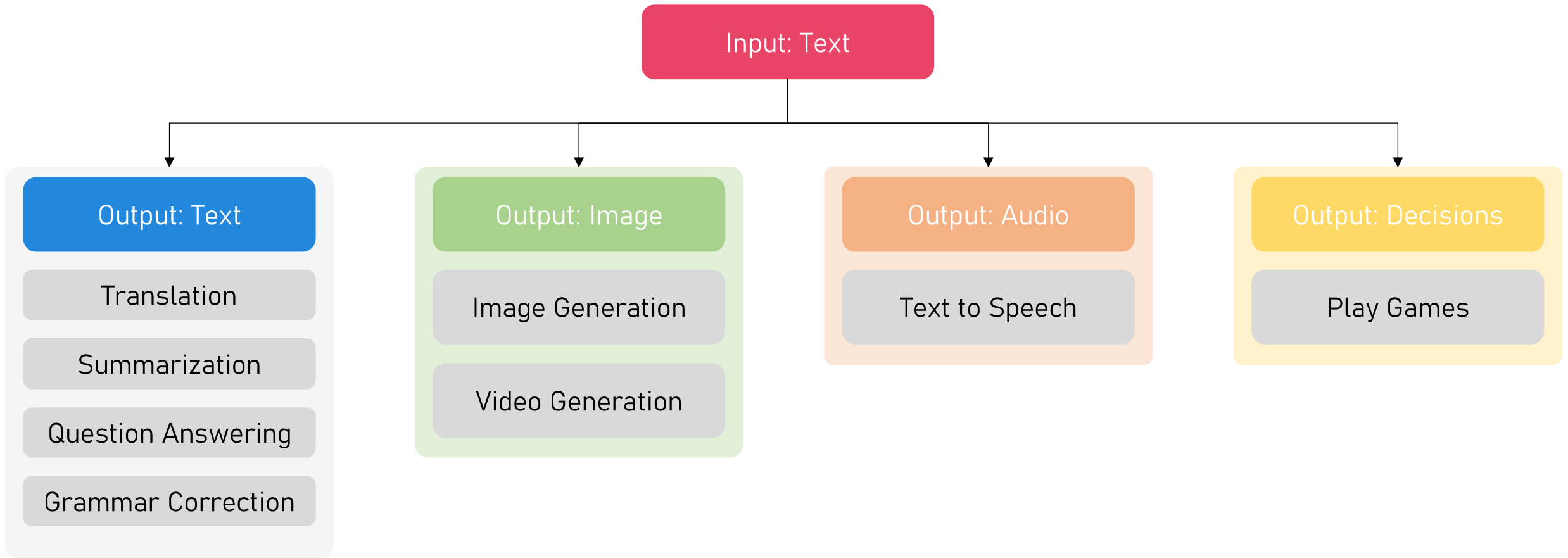
## Generative image models

- Generative image models produce new images using techniques like diffusion.
- Then, given a prompt or related imagery, they transform random noise into images or generate images from prompts.

# Types of Generative AI Based on Data



# Types of Generative AI Based on Data





- OpenAI Codex is a specialized large language model (LLM) designed specifically for code generation and understanding.
- OpenAI Codex is a descendant of GPT-3; its training data contains both natural language and billions of lines of source code from publicly available sources, including code in public GitHub repositories.

# Codex's parameters



## Parameter Count

- Codex has billions of parameters, which allow it to capture intricate coding structures and patterns. While OpenAI hasn't disclosed the exact count, it's assumed to be similar in size to GPT-3, which has 175 billion parameters.

## Language Versatility

- Codex supports multiple programming languages, including Python, JavaScript, TypeScript, Ruby, Go, and more, making it versatile across different development environments.

## Context Handling

- It's designed to understand code-specific context, such as variable names, comments, and function descriptions, making its suggestions context-aware.

## Training Data

- Codex's training on large amounts of public code and text helps it generalize well across common tasks, algorithms, and libraries. This training is crucial to its ability to generate function definitions, entire classes, and even debug code snippets.



## OpenAI Codex Model Card

- Trained on **public code** from GitHub
- Built on top of GPT-3
- Fine-tuned for programming tasks
- Understands over **50 programming languages**
- Powers GitHub Copilot, Copilot Chat, and Copilot Agents
- Learns from code patterns, function usage, and natural language comments
- Supports autocomplete, unit tests, and full-function generation
- Based on transformer architecture
- Uses **multi-modal learning** for code & natural language
- Continuously improved with developer feedback

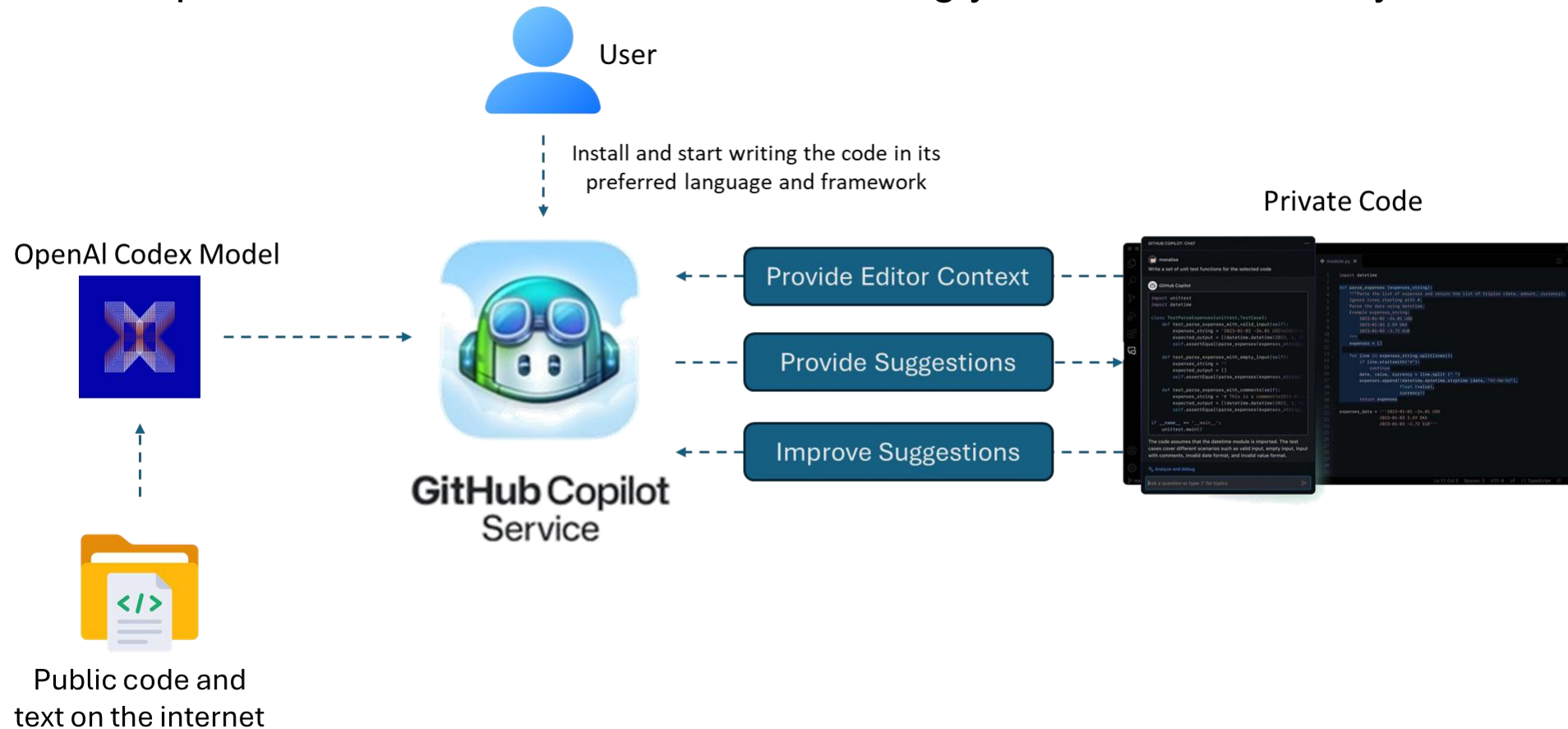


- Derived from **GPT-3**, but optimized for **code tasks**
- Trained on **159+ GB of code** and natural language
- Supports context from **IDE prompts, comments, and usage**
- Responds to both natural language and partial code input
- Used in **VS Code, JetBrains, GitHub.com, and CLI**
- Enables **pair-programming with AI**
- Accepts **docstrings, function names, and comments**
- Powers tools like **Copilot Labs, Code Brushes, and Copilot Agents**

# How GitHub Copilot works



- Copilot makes its suggestions based on code available in its vector library, and uses your code to help with recommendations, without storing your code in its library.



# Other AI Coding Tools



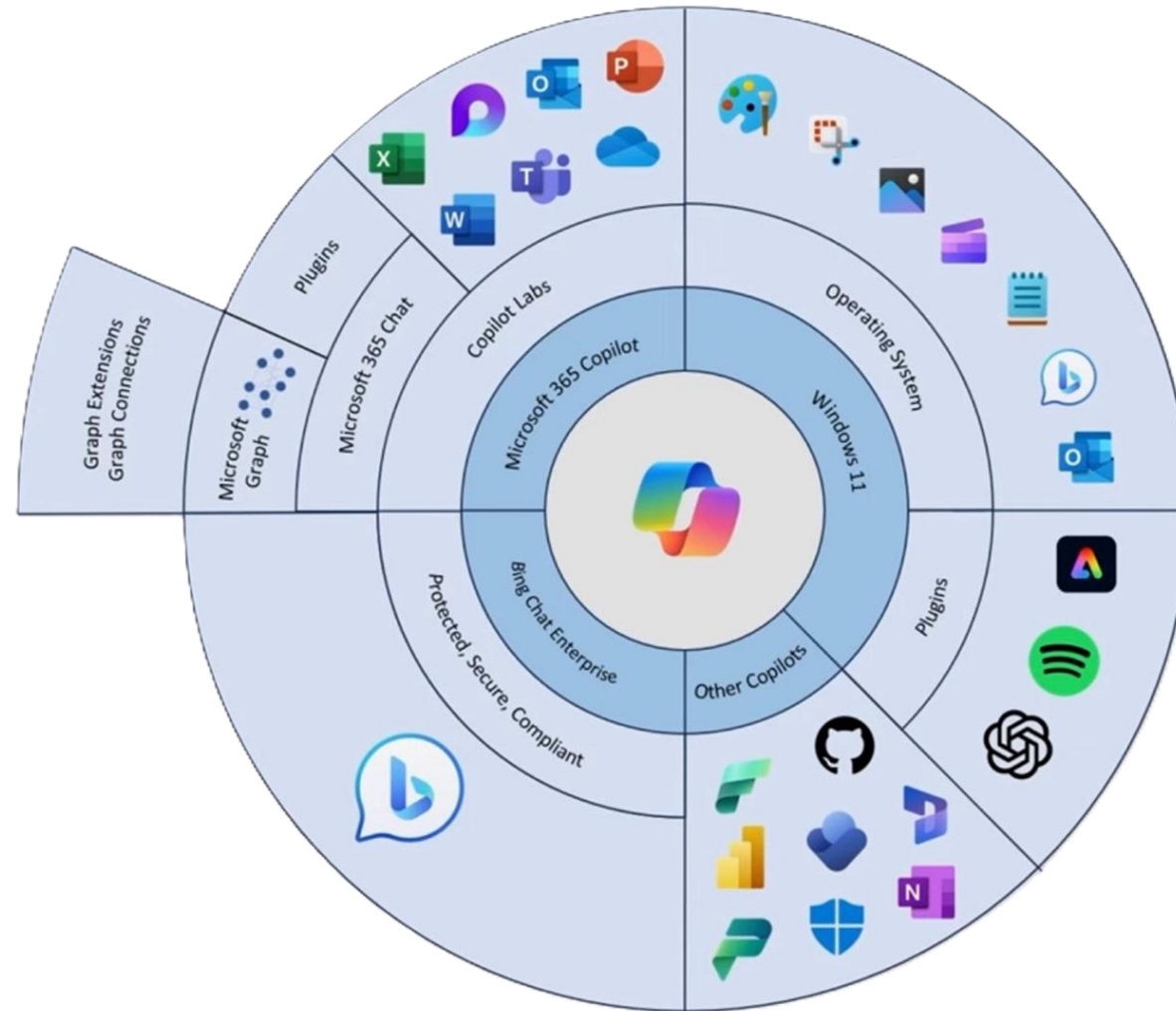
| Feature             | OpenAI Codex                            | Tabnine                   | Amazon CodeWhisperer      |
|---------------------|-----------------------------------------|---------------------------|---------------------------|
| Base Model          | GPT-3 descendant                        | Proprietary               | Proprietary               |
| Languages Supported | 12+ (Python, JavaScript, etc.)          | 20+                       | 15+                       |
| Integration         | GitHub Copilot, OpenAI API, Codex CLI   | IDE plugins               | AWS IDEs                  |
| Unique Strength     | Natural language to code, multi-tasking | Context-aware suggestions | AWS ecosystem integration |
| Cost                | Subscription-based API, Copilot fees    | Free tier, paid plans     | Free for AWS users        |

# Microsoft Copilot



- Microsoft Copilot is an artificial intelligence (AI)-powered chat service that integrates with many other Microsoft applications and services to support user efficiency and productivity.

# Copilot Variants





- The GitHub Copilot Service is an AI/GAI powered code completion tool developed by GitHub in collaboration with OpenAI.

# GitHub Copilot Pricing & Feature

---



# GitHub Copilot Individual

| Feature                      | Free (\$0)     | Pro (\$10/month)       | Pro+ (\$39/month)      |
|------------------------------|----------------|------------------------|------------------------|
| <b>Agents</b>                | Up to 50/month | Unlimited with GPT-4.1 | Unlimited with GPT-4.1 |
| Agent Mode                   | ✓              | ✓                      | ✓                      |
| Code Review                  | ✗              | ✓                      | ✓                      |
| Coding Agent                 | ✗              | ✓                      | ✓                      |
| Copilot Extensions           | ✓              | ✓                      | ✓                      |
| <b>Premium Requests</b>      | 50/month       | 300/month              | 1,500/month            |
| Purchase Additional Requests | ✗              | ✓ (\$0.04/request)     | ✓ (\$0.04/request)     |



# GitHub Copilot Individual

| Feature                               | Free (\$0)    | Pro (\$10/month)       | Pro+ (\$39/month)      |
|---------------------------------------|---------------|------------------------|------------------------|
| <b>Chat Features</b>                  |               |                        |                        |
| Interactions                          | 50/month      | Unlimited with GPT-4.1 | Unlimited with GPT-4.1 |
| Inline Chat                           | ✓             | ✓                      | ✓                      |
| Slash Commands<br>(tests, docs, etc.) | ✓             | ✓                      | ✓                      |
| <b>Available Models</b>               | ✓ (see below) | ✓                      | ✓                      |
| <b>Context-Aware<br/>Coding</b>       | ✓             | ✓                      | ✓                      |
| Debugging/Security<br>Help            | ✓             | ✓                      | ✓                      |
| Open Source<br>Knowledge Access       | ✓             | ✓                      | ✓                      |



# GitHub Copilot Individual

| Feature                         | Free (\$0)  | Pro (\$10/month) | Pro+ (\$39/month) |
|---------------------------------|-------------|------------------|-------------------|
| <b>Chat Features</b>            |             |                  |                   |
| Web Search (Bing)               | ✓           | ✓                | ✓                 |
| Explain Failed Actions Jobs     | ✓           | ✓                | ✓                 |
| Answers on Issues/PRs/Commits   | ✓           | ✓                | ✓                 |
| Multi-file Editing (VS Code)    | ✓           | ✓                | ✓                 |
| <b>Copilot Spaces (Preview)</b> | ✓           | ✓                | ✓                 |
| <b>Code Completion</b>          | 2,000/month | Unlimited        | Unlimited         |
| Next Edit Suggestions           | ✓           | ✓                | ✓                 |
| Comments to Code                | ✓           | ✓                | ✓                 |



# GitHub Copilot Individual

| Feature                                  | Free (\$0) | Pro (\$10/month) | Pro+ (\$39/month) |
|------------------------------------------|------------|------------------|-------------------|
| <b>Customization</b>                     |            |                  |                   |
| Tailored Chat (Private Codebase)         | ✓          | ✓                | ✓                 |
| Unlimited Copilot Extension Integrations | ✓          | ✓                | ✓                 |
| Private Extension Support                | ✓          | ✓                | ✓                 |
| Custom Instructions                      | ✓          | ✓                | ✓                 |
| Knowledge Base Attachments               | ✗          | ✗                | ✗                 |



# GitHub Copilot Individual

| Feature                          | Free (\$0) | Pro (\$10/month) | Pro+ (\$39/month) |
|----------------------------------|------------|------------------|-------------------|
| <b>AI-Native UX</b>              |            |                  |                   |
| Inline Chat & Prompt Suggestions | ✓          | ✓                | ✓                 |
| Slash Commands & Variables       | ✓          | ✓                | ✓                 |
| Commit Message Generation        | ✓          | ✓                | ✓                 |
| PR/Issue/Discussion Summaries    | ✗          | ✓                | ✓                 |
| Code Feedback in VS Code         | ✓          | ✓                | ✓                 |



# GitHub Copilot Individual

| Feature                               | Free (\$0) | Pro (\$10/month) | Pro+ (\$39/month) |
|---------------------------------------|------------|------------------|-------------------|
| <b>AI-Native UX</b>                   |            |                  |                   |
| Visual Studio Quick Info Explanations | ✓          | ✓                | ✓                 |
| Debug Assistant (Visual Studio)       | ✓          | ✓                | ✓                 |
| Java Upgrade Assistant (VS Code)      | ✗          | ✓                | ✓                 |
| <b>Supported Environments</b>         |            |                  |                   |
| Editors & IDEs                        | ✓          | ✓                | ✓                 |
| GitHub.com                            | ✓          | ✓                | ✓                 |
| GitHub Mobile                         | ✓          | ✓                | ✓                 |
| GitHub CLI & Terminal                 | ✓          | ✓                | ✓                 |



# GitHub Copilot Individual

| Feature                          | Free (\$0) | Pro (\$10/month) | Pro+ (\$39/month) |
|----------------------------------|------------|------------------|-------------------|
| <b>Management &amp; Security</b> |            |                  |                   |
| Public Code Filter               | ✓          | ✓                | ✓                 |
| User Management                  | ✗          | ✗                | ✗                 |
| Data Excluded from Training      | ✓          | ✓                | ✓                 |
| Enterprise-Grade Security        | ✗          | ✗                | ✗                 |
| IP Indemnity                     | ✗          | ✗                | ✗                 |
| Content Exclusions               | ✗          | ✗                | ✗                 |
| SAML SSO Auth                    | ✗          | ✗                | ✗                 |
| Usage Metrics                    | ✗          | ✗                | ✗                 |
| Requires GitHub Enterprise Cloud | ✗          | ✗                | ✗                 |

# GitHub Copilot Business and Enterprise



| Feature            | Business (\$19/user/month) | Enterprise (\$39/user/month) |
|--------------------|----------------------------|------------------------------|
| <b>Agents</b>      |                            |                              |
| Agent Mode         | Unlimited with GPT-4.1     | Unlimited with GPT-4.1       |
| Code Review        | ✓                          | ✓                            |
| Coding Agent       | ✓                          | ✓                            |
| Copilot Extensions | ✓                          | ✓                            |

# GitHub Copilot Business and Enterprise



| Feature                            | Business (\$19/user/month) | Enterprise (\$39/user/month) |
|------------------------------------|----------------------------|------------------------------|
| <b>Premium Requests</b>            |                            |                              |
| Monthly Premium Requests           | 300/user                   | 1,000/user                   |
| Purchase Additional Requests       | ✓ (\$0.04/request)         | ✓ (\$0.04/request)           |
| <b>Chat Features</b>               |                            |                              |
| Chat Interactions                  | Unlimited with GPT-4.1     | Unlimited with GPT-4.1       |
| Inline Chat                        | ✓                          | ✓                            |
| Slash Commands (tests, docs, etc.) | ✓                          | ✓                            |
| <b>Available Chat Models</b>       | ✓ (see below)              | ✓ (see below)                |

# GitHub Copilot Business and Enterprise



| Feature                               | Business (\$19/user/month) | Enterprise (\$39/user/month) |
|---------------------------------------|----------------------------|------------------------------|
| <b>Coding Assistance</b>              |                            |                              |
| Context-Aware Support & Explanations  | ✓                          | ✓                            |
| Debugging & Security Help             | ✓                          | ✓                            |
| Access to Open Source Knowledge       | ✓                          | ✓                            |
| Web Search (Bing)                     | ✓                          | ✓                            |
| Explain Failed Actions Jobs           | ✓                          | ✓                            |
| Answers on Issues, PRs, Commits, etc. | ✓                          | ✓                            |
| Multi-file Editing (VS Code)          | ✓                          | ✓                            |
| <b>Copilot Spaces (Preview)</b>       | ✓                          | ✓                            |

# GitHub Copilot Business and Enterprise



| Feature                                  | Business (\$19/user/month) | Enterprise (\$39/user/month) |
|------------------------------------------|----------------------------|------------------------------|
| <b>Code Completion</b>                   |                            |                              |
| GPT-4o Real-Time Suggestions             | Unlimited                  | Unlimited                    |
| Next Edit Suggestions                    | ✓                          | ✓                            |
| Comments to Code                         | ✓                          | ✓                            |
| <b>Customization</b>                     |                            |                              |
| Tailored Chat to Private Codebase        | ✓                          | ✓                            |
| Unlimited Copilot Extension Integrations | ✓                          | ✓                            |
| Private Extensions (Internal Tools)      | ✓                          | ✓                            |
| Custom Instructions                      | ✓                          | ✓                            |
| Attach Knowledge Bases to Chat           | ✗                          | ✓                            |

# GitHub Copilot Business and Enterprise



| Feature                            | Business (\$19/user/month) | Enterprise (\$39/user/month) |
|------------------------------------|----------------------------|------------------------------|
| <b>AI-Native Experiences</b>       |                            |                              |
| Inline Chat & Prompt Suggestions   | ✓                          | ✓                            |
| Slash Commands & Context Variables | ✓                          | ✓                            |
| Commit Message Generation          | ✓                          | ✓                            |
| PR/Issue/Discussion Summaries      | ✓                          | ✓                            |
| Code Feedback in VS Code           | ✓                          | ✓                            |
| Quick Info Explanations (VS)       | ✓                          | ✓                            |
| Debug Assistant (VS)               | ✓                          | ✓                            |
| Java Upgrade Assistant (VS Code)   | ✓                          | ✓                            |

# GitHub Copilot Business and Enterprise



| Feature                       | Business (\$19/user/month) | Enterprise (\$39/user/month) |
|-------------------------------|----------------------------|------------------------------|
| <b>Supported Environments</b> |                            |                              |
| Editors & IDEs                | ✓                          | ✓                            |
| GitHub.com                    | ✓                          | ✓                            |
| GitHub Mobile                 | ✓                          | ✓                            |
| GitHub CLI & Windows Terminal | ✓                          | ✓                            |

# GitHub Copilot Business and Enterprise



| Feature                          | Business (\$19/user/month) | Enterprise (\$39/user/month) |
|----------------------------------|----------------------------|------------------------------|
| <b>Management &amp; Policies</b> |                            |                              |
| Public Code Filter               | ✓                          | ✓                            |
| User Management                  | ✓                          | ✓                            |
| Data Exclusion from Training     | ✓                          | ✓                            |
| Enterprise-Grade Security        | ✓                          | ✓                            |
| IP Indemnity                     | ✓                          | ✓                            |
| Content Exclusions               | ✓                          | ✓                            |
| SAML SSO Authentication          | ✓                          | ✓                            |
| Usage Metrics                    | ✓                          | ✓                            |
| Requires GitHub Enterprise Cloud | ✗                          | ✗                            |

# Test of Productivity with Copilot



- Conducted a test with developers in 2 offices and saw a 20-50% productivity gain from using Copilot

## Experiment:

- 4 Pods, 2 in Mumbai, 2 in Bangalore
- Identical code bases & backlogs
- Half used GitHub Copilot, half did not
- Developers had no training or previous experience on the codebase or on GitHub Copilot
- Pods were given 1 Sprint to complete the work

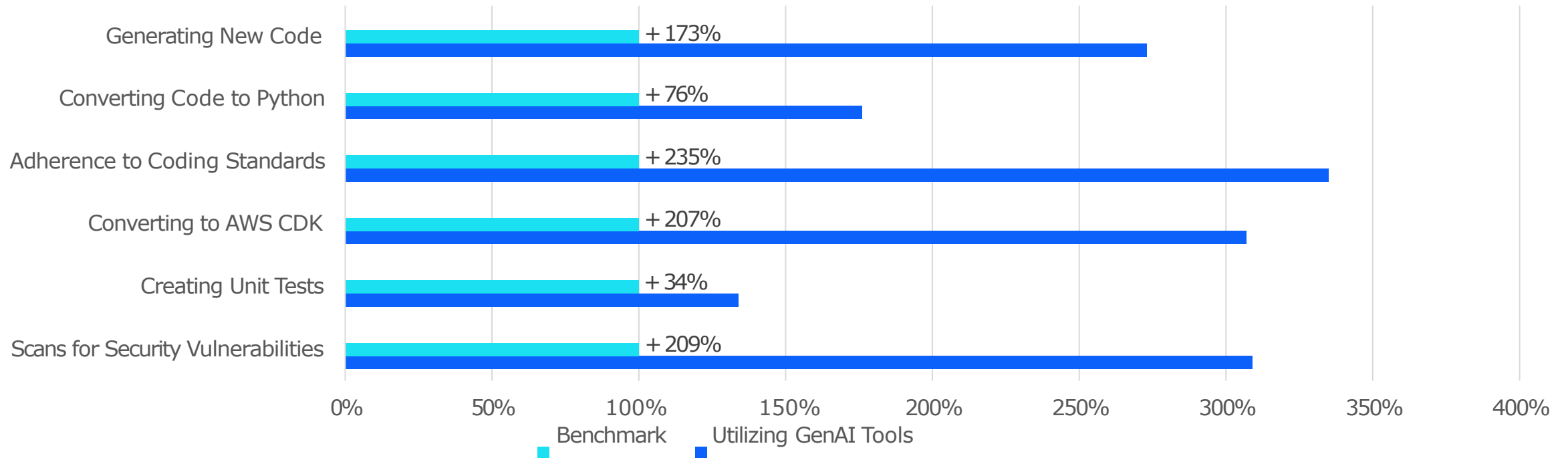
## What we Saw:

- **20-50% faster code production** with Copilot
- Less experienced engineers saw a bigger productivity boost
- Code Quality varied by language
- Led to more unnecessary code duplication in some languages

# Massive Productivity Gains with GenAI Tooling



Observed Productivity Increase by Use Case (# of Lines of Code Added or Deleted Per Minute)



*I found it beneficial not having to shift back and forth with the browser (Google, Stack Overflow), it was able to find good answers quickly, and can read the context of the question*

*Allows developers to come up to speed faster because the code is standardized and uniform*

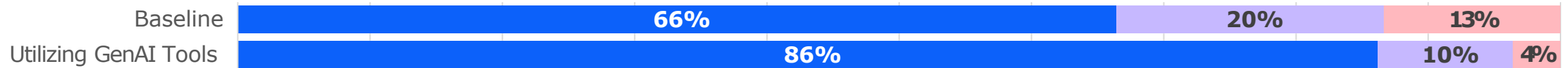
*The tools enables Developers to quickly convert between languages without requiring a deep understanding of both languages, allowing much faster time to productivity*

# While Improving Developer Satisfaction

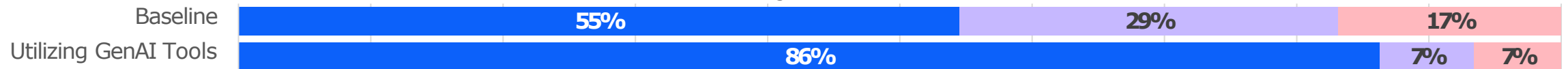


Developer satisfaction with key areas of job before and after pilot (averages across all use cases)

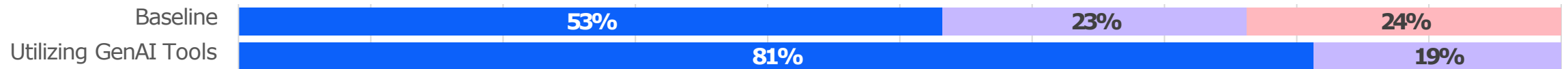
## Productivity Satisfaction



## Code Quality Satisfaction



## Standardization Satisfaction



## Flow State Satisfaction



## Overall Satisfaction



■ (Strongly) Satisfied   ■ Neutral   ■ (Strongly) Dissatisfied

# Focus on Problem Solving



- Developers using GitHub Copilot:

**Spend More Time:**

- Thinking about how to structure code to solve business problems
- Writing pseudo-code and code comments of the problems they are solving
- Reviewing and tweaking AI-generated code

**Spend Less Time:**

- Writing boilerplate and repetitive code
- Searching google and Stack Overflow

“I can spend more time focusing on solving and writing complex business logic and less time writing unit tests”

# What It's Like as a Developer



- Code suggestions right in your IDE, with the context of your IDE.

```
J main.java
1 public static void CreateTables()
2 {
3     using (var context = new TaskContext())
4     {
5         context.Database.ExecuteSqlCommand("CREATE TABLE Tasks (id INT PRIMARY KEY, name VARCHAR(50), priority INT, status VARCHAR(50))");
6     }
7 }
8
9 // Generate a function that retrieves DB items by priority
10 public static void GetTasksByPriority(int priority)
11 {
12     using (var context = new TaskContext())
13     {
14         var tasks = context.Tasks.Where(t => t.Priority == priority).ToList();
15         foreach (var task in tasks)
16         {
17             Console.WriteLine(task.Name);
18         }
19     }
20 }
```



# What is AI-Powered Code Generation?

- Code Generation Powered by GitHub Copilot and OpenAI
  - AI-driven suggestions for code completion
  - Generate code in real time based on context and patterns
  - Single, Specific and Short prompts
  - Automatically draft code comments based on your code
  - No need to context-switch while developing



# Code Exposure Outside & Client Environments

- While this is a common concern for all Gen AI technologies, GitHub Copilot has it covered!
- GitHub Copilot transmits snippets of your code from your IDE to the GitHub Copilot backend in the form of a prompt to provide suggestions to you.
- Code snippets data is only transmitted in real-time to return suggestions and is discarded once suggestions are returned.
- Copilot for Business does NOT retain any code snippets data.
- Copilot for Business does NOT use your code as training data for the underlying Large Language Model

# Safety of Executing Generated Code



- Code is generated based on open source in GitHub and available elsewhere on the web, using your IDE's context to make suggestions specific. There is no guarantee that it works or is safe



## Copilot Doesn't Replace Developers

- Like with code found on Stack Overflow, you shouldn't run it if you don't understand it
- Developers need to understand their code, Copilot is just here to help them be more productive and learn more quickly



## Testing & Reviews are a Necessity

- Software Development Practices should still be followed: All code is unit tested and QE'd, and peer-reviewed before merged
- All Software Copilot training emphasizes this as a core requirement to usage!



# Filtering Mechanism to Mitigate Risk

- GitHub Copilot includes an optional code referencing filter to detect and suppress certain suggestions that match public code on GitHub.
  - GitHub has created a duplication detection filter to detect and suppress suggestions that contain code segments over a certain length that match public code on GitHub. This filter can be enabled by the administrator for your enterprise and it can apply for all organizations within your enterprise, or the administrator can defer control to individual organizations.
  - With the filter enabled, Copilot checks code suggestions for matches or near-matches against public code on GitHub of 65 lexemes or more (on average, 150 characters). If there is a match, the suggestion will not be shown to the user.
- In addition to off-topic, harmful, and offensive output filters, GitHubCopilot also scans the outputs for vulnerable code

# Copilot for Business: Security and Privacy



- GitHub Copilot for Business is an AI pair-programmer that helps developers write code faster with less work. GitHub Copilot is powered by GitHub customized variants of OpenAI's models, which are generative pretrained Large Language Models optimized for code assistance.
- GitHub is committed to transparency around privacy and security of all their products.
- Copilot for Business:
  - Does NOT retain user content (e.g. code from user's editor)
  - Does NOT use your code to Improve the model
  - Does NOT retain suggestions produced by Copilot
  - Does NOT send information to OpenAI, the software company
  - Is governed by both GitHub Enterprise and Copilot's product specific terms

# GitHub as Data Controller



Subject to authorization through GitHub's Data Protection Agreement (DPA), GitHub processes UED as a data controller only for following purposes:

- Billing and account management
- Compensation, such as calculating employee commissions and partner incentives
- Aggregated internal reporting and business modeling, such as forecasting, revenue, capacity planning, and product strategy
- Aggregated financial reporting

For both its controller and processor roles, GitHub applies the principles of data minimization.

## Sources of Suggestions

- GitHub Copilot is powered by OpenAI Large Language Models (LLMs). They are trained on natural language text and source code from publicly available sources. OpenAI does not disclose its training set or specify all of its sources, however, the models are not trained on any code from private repositories.
- GitHub Copilot does not "copy and paste" code used to train its model - rather, it generates new code in a probabilistic way, and the probability that a suggestion contains the same code as a snippet utilized in training is low. Previous research showed that many of these cases happen when GitHub Copilot is unable to glean sufficient context from the code you are writing, or when there is a common, perhaps even universal, solution to the problem.
- GitHub Copilot also checks whether code suggestions match code in GitHub public repositories. Any suggestions matching at 150 characters or more (excluding whitespace) will be flagged. Organization owners and admins can elect to have such matches blocked before they are shown.

# GitHub as Data Controller



## 6. Data.

- A. **Generally.** GitHub Copilot sends an encrypted Prompt from you to GitHub to provide Suggestions to you. Except as detailed below, Prompts are transmitted only to generate Suggestions in real-time, are deleted once Suggestions are generated, and are not used for any other purpose. Prompts are encrypted during transit and are not stored at rest without your permission.
- B. **When Prompts are Retained.** Your Prompts are retained by GitHub in the following circumstances:
  - 1. *CLI and Other Tools.* If you use GitHub Copilot tools that operate outside of your code editor, such as Copilot for the Command Line Interface, GitHub Copilot retains your Prompts to those tools to provide the service.
  - 2. *Private Language Models.* If you have requested a customized private language model, GitHub Copilot retains your Prompts to fine-tune your private model.
  - 3. *Custom Settings.* If you have configured GitHub Copilot to use alternative data handling, such as enabling interaction with third party extensions, GitHub Copilot will retain your Prompts based on that configuration.
- C. **More Information.** More detailed information on how data is processed by GitHub Copilot is available at [gh.io/privacy](https://github.com/privacy).

# How we're using AI to accelerate product engineering



## Backlog refinement and planning

- Automate backlog management, prioritizing items based on complexity and business value
- Predict story points for new user stories using historical data
- Enhance sprint planning with an optimal mix of user stories for each sprint based on complexity and team velocity

## Design and develop

- Provide large blocks of code from natural language requests
- Add to code written through copilot functionality
- Analyze and understand technical debt, refactor and simplify complex solutions, and generate code and templates

## Testing

- Generate unit and integration tests
- Identify and create code to test security vulnerabilities
- Generate synthetic data for data platform modernization programs
- Automate code reviews
- Predict potential system failures or bugs
- Create rapid prototypes

# GitHub Copilot Service



- The GitHub Copilot Service is an AI-powered code completion tool developed by GitHub in collaboration with OpenAI. Here's a breakdown of its key aspects:
  - Real-Time Code Suggestions
  - IDE Integration
  - Contextual Understanding
  - Adaptability
  - Support for Multiple Languages

# Getting Started with GitHub Copilot

# Supported Development Environment



## Supported IDEs and Extensions

Available as an extension in:

- Visual Studio Code
- Visual Studio
- Vim
- Neovim
- JetBrains suite of IDEs
- Azure Data Studio

## Functionality

- Code completion functionality is available across all extensions.
- Chat functionality is available only in:
  - Visual Studio Code
  - JetBrains suite
  - Visual Studio

## Terminal Support

- Supported in terminals via:
  - GitHub CLI
  - Chat integration in Windows Terminal Canary

## GitHub Mobile Support

- All plans supported in GitHub Copilot in GitHub Mobile.
- Copilot Individual and Copilot Business:
  - Access to Bing and public repository code search.
- Copilot Enterprise in GitHub Mobile:
  - Additional access to your organization's knowledge.

# Set up and integrate Copilot into your IDE



- let's dive in and see a live demonstration of setting up and using GitHub Copilot in Visual Studio!



# Set up and integrate Copilot into your CLI

- Make sure you have installed gh in your system
- ``gh extension install github/gh-copilot`` to install github copilot in your terminal
- ``gh extension upgrade gh-copilot`` to upgrade your github copilot.

# Writing Efficient Code demo

# Objectives



Demonstrate how GitHub Copilot speeds up the development process and in what specific areas it provides the most value.



Provide insights about what GitHub Copilot does – and does not do - through best practice usage considerations.



Understanding the recommended deployment approach for GitHub Copilot in your company factoring in comms & team engagement.

# Ways to Interact with Copilot



# Code completions



- press “tab” to accept a suggestion.

```
78     }
79
80     3 references
    public async Task DeleteBasketAsync(int basketId)
81     {
82         I
83     }
84 }
85
```

# Inline chat



- Use "/" to state your intent or "#" to refer to your files(s). Like "/test for #filename"

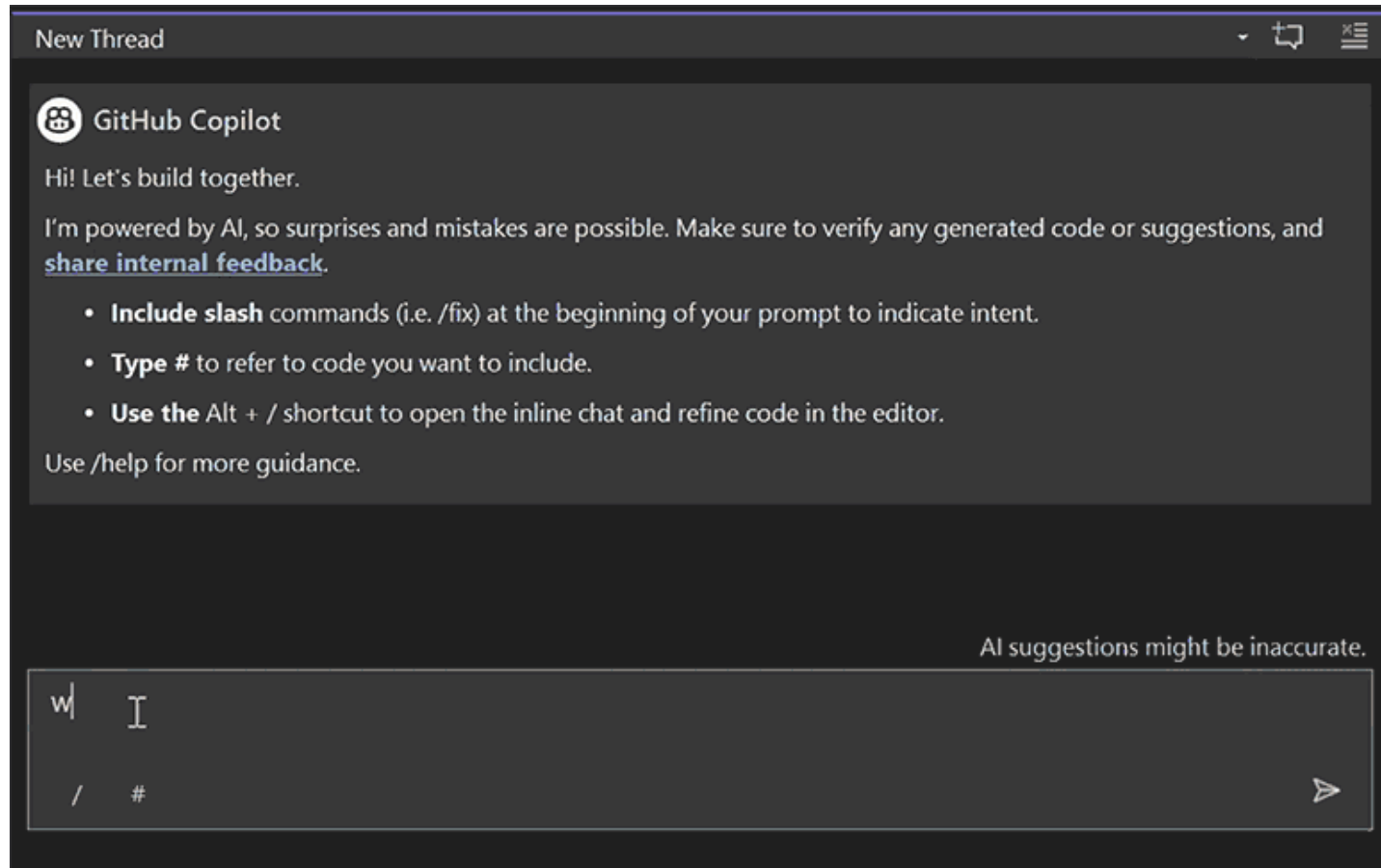
```
90
91
92
93
94
95
96
97
98
99
100
101
```

```
if(userName == null)
{
    userName = Guid.NewGuid().ToString();
    Response.Cookies.Append(Constants.BASKET_COOKIE_NAME, userName, new CookieOptions
    {
        Expires = DateTime.Today.AddYears(10)
    });
}
```

# Chat window



- Use #solution to refer to your active solution (only for C#) in Visual Studio

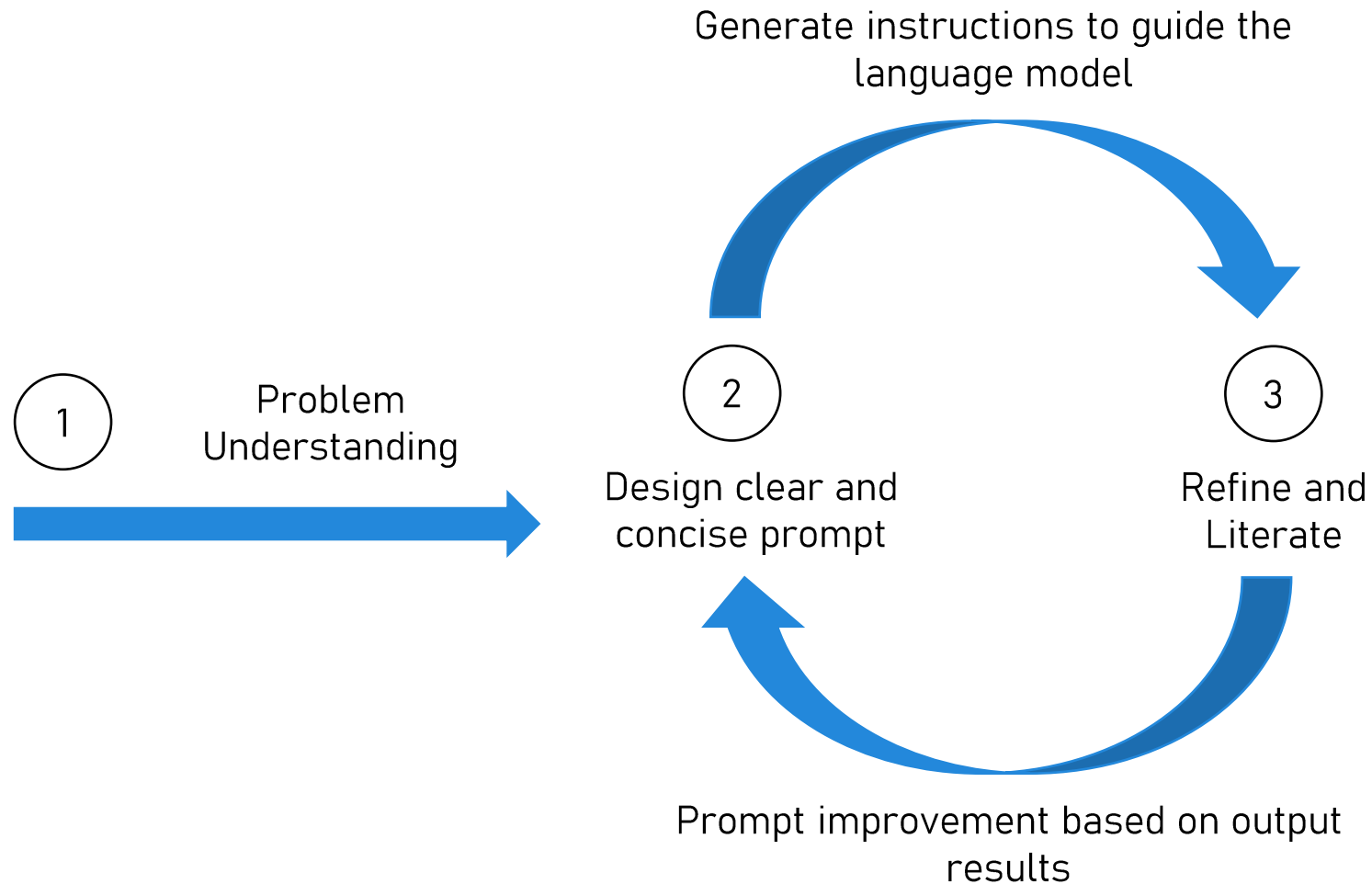


# Prompt engineering

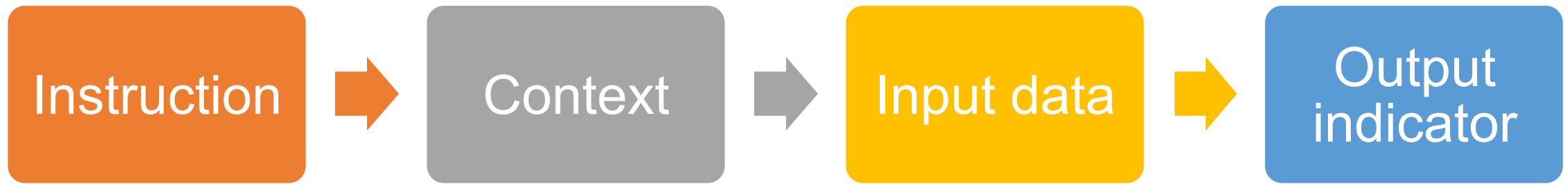


- **Definition:** Crafting & refining prompts for models like GPT-3 to produce text.
- **Objective:** Enhance output quality & relevancy.
- **Application:** Chatbots, translation, content generation, etc.
- **Process:**
  - Choose model architecture & parameters
  - Design prompt structure & format
  - Select task & training data
  - Fine-tune model with prompt & data

# Prompt engineering



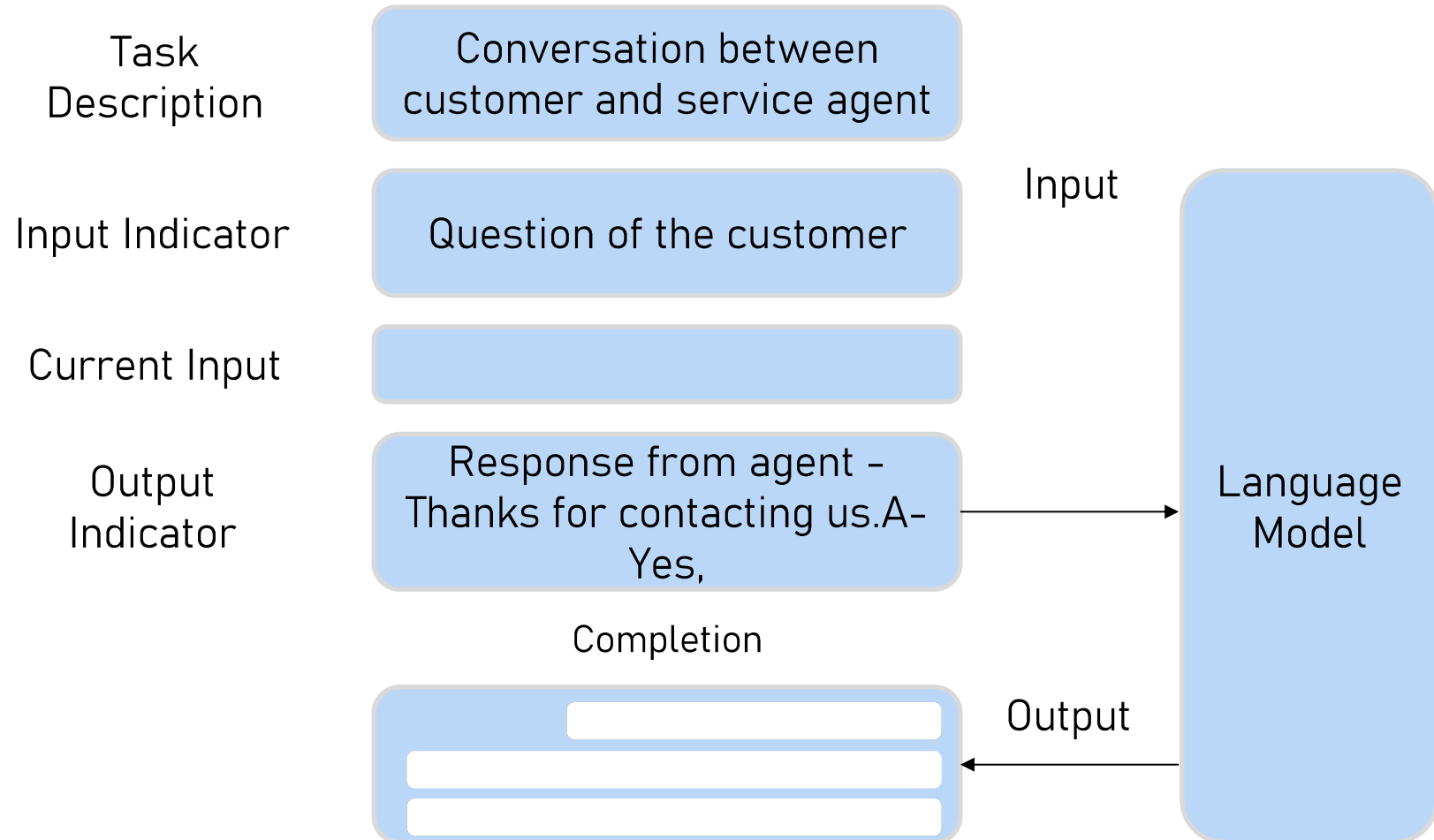
# Elements of a Prompt



# Basic of Prompting



## Prompt:



# Examples of Prompts



# Examples of Prompts

- Text Summarization
- Information Extraction
- Question Answering
- Text Classification
- Conversation
- Code Generation
- Reasoning

# Text Summarization



- One of the standard tasks in natural language generation is text summarization. Text summarization can include many different flavors and domains. In fact, one of the most promising applications of language models is the ability to summarize articles and concepts into quick and easy-to-read summaries.
- Known Use Cases:
  - Email Summarization
  - Report Summarization
  - Meeting Summarization
  - Etc...

# Text Summarization



Antibiotics are a type of medication used to treat bacterial infections. They work by either killing the bacteria or preventing them from reproducing, allowing the body's immune system to fight off the infection. Antibiotics are usually taken orally in the form of pills, capsules, or liquid solutions, or sometimes administered intravenously. They are not effective against viral infections, and using them inappropriately can lead to antibiotic resistance.

Context



Explain the above in one sentence.

Instruction

Antibiotics are medications used to treat bacterial infections by either killing the bacteria or stopping them from reproducing, but they are not effective against viruses and overuse can lead to antibiotic resistance.



# Information Extraction



- While language models are trained to perform natural language generation and related tasks, it's also very capable of performing classification and a range of other natural language processing (NLP) tasks.
- Known Use Cases:
  - Image Captioning
  - Automatic Form filling
  - Text-to-Speech
  - Speech-to-Text
  - Optical Character Recognition
  - Etc...

# Information Extraction



Author-contribution statements and acknowledgements in research papers should state clearly and specifically whether, and to what extent, the authors used AI technologies such as **ChatGPT** in the preparation of their manuscript and analysis. They should also indicate which LLMs were used. This will alert editors and reviewers to scrutinize manuscripts more carefully for potential biases, inaccuracies and improper source crediting. Likewise, scientific journals should be transparent about their use of LLMs, for example when selecting submitted manuscripts.

**Mention the large language model based product mentioned in the paragraph above:**

The large language model based product mentioned in the paragraph above is **ChatGPT**.



# Question Answering



- One of the best ways to get the model to respond with specific answers is to improve the format of the prompt. As covered before, a prompt could combine instructions, context, input, and output indicators to get improved results. While these components are not required, it becomes a good practice as the more specific you are with instruction, the better results you will get.
- Known Use Cases:
  - Chatbot
  - Customer Support
  - Virtual Assistants
  - Etc...

# Question Answering



**Answer the question based on the context below. Keep the answer short and concise. Respond "Unsure about answer" if not sure about the answer..**

**Context:** Teplizumab traces its roots to a New Jersey drug company called Ortho Pharmaceutical. There, scientists generated an early version of the antibody, dubbed OKT3. Originally sourced from mice, the molecule was able to bind to the surface of T cells and limit their cellkilling potential. In 1986, it was approved to help prevent organ rejection after kidney transplants, making it the first therapeutic antibody allowed for human use.

**Question:** What was OKT3 originally sourced from?

**Answer:** Mice.



# Text Classification



- So far, you have used simple instructions to perform a task. As a prompt engineer, you need to get better at providing better instructions. But that's not all! You will also find that for harder use cases, just providing instructions won't be enough. This is where you need to think more about the context and the different elements you can use in a prompt. Other elements you can provide are input data or examples.
- Known Use Cases:
  - Sentiment Analysis
  - Spam Detection
  - Language Detection
  - Etc...

# Text Classification



**Classify the text into neutral, negative or positive.**

**Text:** I think the food was okay.

**Sentiment:** Neutral





- Perhaps one of the more interesting things you can achieve with prompt engineering is instructing the LLM system on how to behave, its intent, and its identity. This is particularly useful when you are building conversational systems like customer service chatbots.
- Known Use Cases:
  - Booking and Reservations
  - Event Planning
  - Technical Support
  - Etc...

# Conversation



**The following is a conversation with an AI research assistant. The assistant tone is technical and scientific.**



Hello, who are you?



Can you tell me about the creation of blackholes?

Sure! Black holes are regions of spacetime where the gravitational force is so strong that nothing, not even light, can escape from it. They are created when a very massive star dies and its core collapses in on itself, forming a singularity of infinite density. The intense gravity of the singularity pulls in all the matter and radiation around it, creating the black hole..



# Code Generation



- One application where LLMs are quite effective is code generation. Copilot is a great example of this. There are a vast number of code-generation tasks you can perform with clever prompts.
- Known Use Cases:
  - Code Completion
  - Bug Fixing
  - Refactoring Code
  - Etc...

# Code Generation



*Table departments, columns = [DepartmentId, DepartmentName]  
Table students, columns = [DepartmentId, StudentId, StudentName]  
Create a MySQL query for all students in the Computer Science Department*

```
SELECT StudentId, StudentName  
FROM students  
WHERE DepartmentId IN (SELECT DepartmentId FROM departments WHERE  
DepartmentName = 'Computer Science');
```



# Reasoning



- Perhaps one of the most difficult tasks for an LLM today is one that requires some form of reasoning. Reasoning is one of most interesting areas due to the types of complex applications that can emerge from LLMs.
- Known Use Cases:
  - Logical Reasoning
  - Mathematical Reasoning
  - Planning and Decision Making
  - Etc...

# Reasoning



The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.  
**Solve by breaking the problem into steps. First, identify the odd numbers, add them, and indicate whether the result is odd or even.**

Odd numbers: 15, 5, 13, 7, 1  
Sum: 41  
41 is an odd number.



# Prompt Engineering Techniques



# Prompt Engineering Techniques

- Zero shot
- Few shot
- Chain of Thought (CoT)
- Self consistency
- Generated Knowledge Prompting
- Prompt Chaining
- Tree of Thoughts(ToT)
- Automatic Reasoning and Tool use (ART)
- Directional Stimulus
- ReAct Prompting

# Zero shot



- Zero-shot prompting is a powerful technique where you can use a pre-trained language model to perform various tasks **without any fine-tuning**.
- This technique can be applied to various tasks, such as translation, summarization, question answering, and more.

# Zero shot



Classify the text into neutral, negative or positive.

**Text:** I think the vacation is okay.

**Sentiment:**

**Sentiment:** Neutral



# Few shot

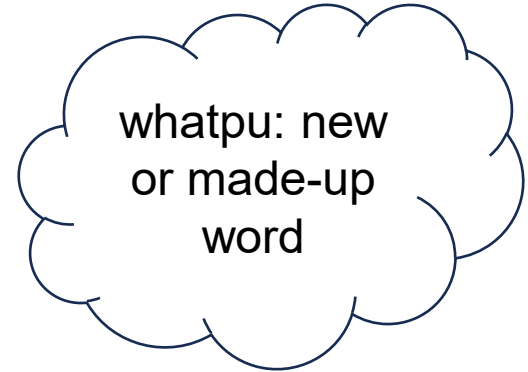


- Few-shot prompting is a technique that builds upon the principles of zero-shot prompting but provides the model **with a few examples** to give it a better understanding of the desired task.
- Instead of relying solely on its pre-existing knowledge, the model is guided by a small number of input-output pairs.

# Few shot Example 1



A "**whatpu**" is a small, furry animal native to Tanzania.  
**An example of a sentence that uses the word whatpu is:**  
We were traveling in Africa and we saw these very cute whatpus.



To do a "**farduddle**" means to jump up and down really fast.  
**An example of a sentence that uses the word farduddle is:**

When we won the game, we all started to farduddle in celebration

farduddle:  
new or  
made-up  
word



# Few shot Example 2



This is awesome! // **Positive**  
This is bad! // **Negative**  
Wow that movie was great! // **Positive**  
What a horrible show! //

**Negative**





# Limitations of Few-shot Prompting

- Standard few-shot prompting works well for many tasks but is still not a perfect technique, especially when dealing with more complex reasoning tasks.

# Limitations of Few-shot Prompting



The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.  
A:

Yes, the odd numbers in this group add up to 107, which is an even number. ❌



# Limitations of Few-shot Prompting



The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.

A: The answer is False.

The odd numbers in this group add up to an even number: 17, 10, 19, 4, 8, 12, 24.

A: The answer is True.

The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13, 24.

A: The answer is True.

The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2.

A: The answer is False.

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.

A:

The answer is True..



# Chain of Thought (CoT)



- Chain-of-Thought (CoT) prompting is a technique that involves generating a sequence of prompts in a conversation-like format to guide an AI model's responses.
- Each prompt builds upon the context and information provided by the previous prompts, allowing for more coherent and focused interactions.

# Standard vs Chain of Thought



## Standard Prompting



Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?



A: The answer is 27. ❌

## Chain-of-Thought Prompting



Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls.  $5 + 6 = 11$ . The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had  $23 - 20 = 3$ . They bought 6 more apples, so they have  $3 + 6 = 9$ .

The answer is 9. ✅



# Chain of Thought Example 1:



The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.

A: Adding all the odd numbers (9, 15, 1) gives 25. The answer is False.

The odd numbers in this group add up to an even number: 17, 10, 19, 4, 8, 12, 24.

A: Adding all the odd numbers (17, 19) gives 36. The answer is True.

The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13, 24.

A: Adding all the odd numbers (11, 13) gives 24. The answer is True.

The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2.

A: Adding all the odd numbers (17, 9, 13) gives 39. The answer is False.

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.

A:

Adding all the odd numbers (15, 5, 13, 7, 1) gives 41. The answer is False.



# Chain of Thought Example 2:



The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.  
A: Adding all the odd numbers (9, 15, 1) gives 25. The answer is False.

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.  
A:

Adding all the odd numbers (15, 5, 13, 7, 1) gives 41. The answer is False..



# Self consistency



- Self-Consistency Prompting is a technique that involves repeatedly asking an AI model to provide examples or correct its own outputs.
- By guiding the model to improve its responses based on its own previous answers, you can enhance the accuracy and consistency of generated content.

# Self consistency Example:



When I was 6 my sister was half my age. Now I'm 70 how old is my sister?

35 ❌



# Self consistency Example:



Q: There are 15 trees in the grove. Grove workers will plant trees in the grove today. After they are done, there will be 21 trees. How many trees did the grove workers plant today?

A: We start with 15 trees. Later we have 21 trees. The difference must be the number of trees they planted. So, they must have planted  $21 - 15 = 6$  trees. The answer is 6.

Q: If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?

A: There are 3 cars in the parking lot already. 2 more arrive. Now there are  $3 + 2 = 5$  cars. The answer is 5.

Q: Leah had 32 chocolates and her sister had 42. If they ate 35, how many pieces do they have left in total?

A: Leah had 32 chocolates and Leah's sister had 42. That means there were originally  $32 + 42 = 74$  chocolates. 35 have been eaten. So in total they still have  $74 - 35 = 39$  chocolates. The answer is 39.

Q: Jason had 20 lollipops. He gave Denny some lollipops. Now Jason has 12 lollipops. How many lollipops did Jason give to Denny?

A: Jason had 20 lollipops. Since he only has 12 now, he must have given the rest to Denny. The number of lollipops he has given to Denny must have been  $20 - 12 = 8$  lollipops. The answer is 8.

# Self consistency Example:



Q: When I was 6 my sister was half my age. Now I'm 70 how old is my sister?  
A:

When I was 6 my sister was half my age, so she was 3. Now I am 70, so she is  $70 - 3 = 67$ . The answer is 67.



# CoT vs Self-Consistency



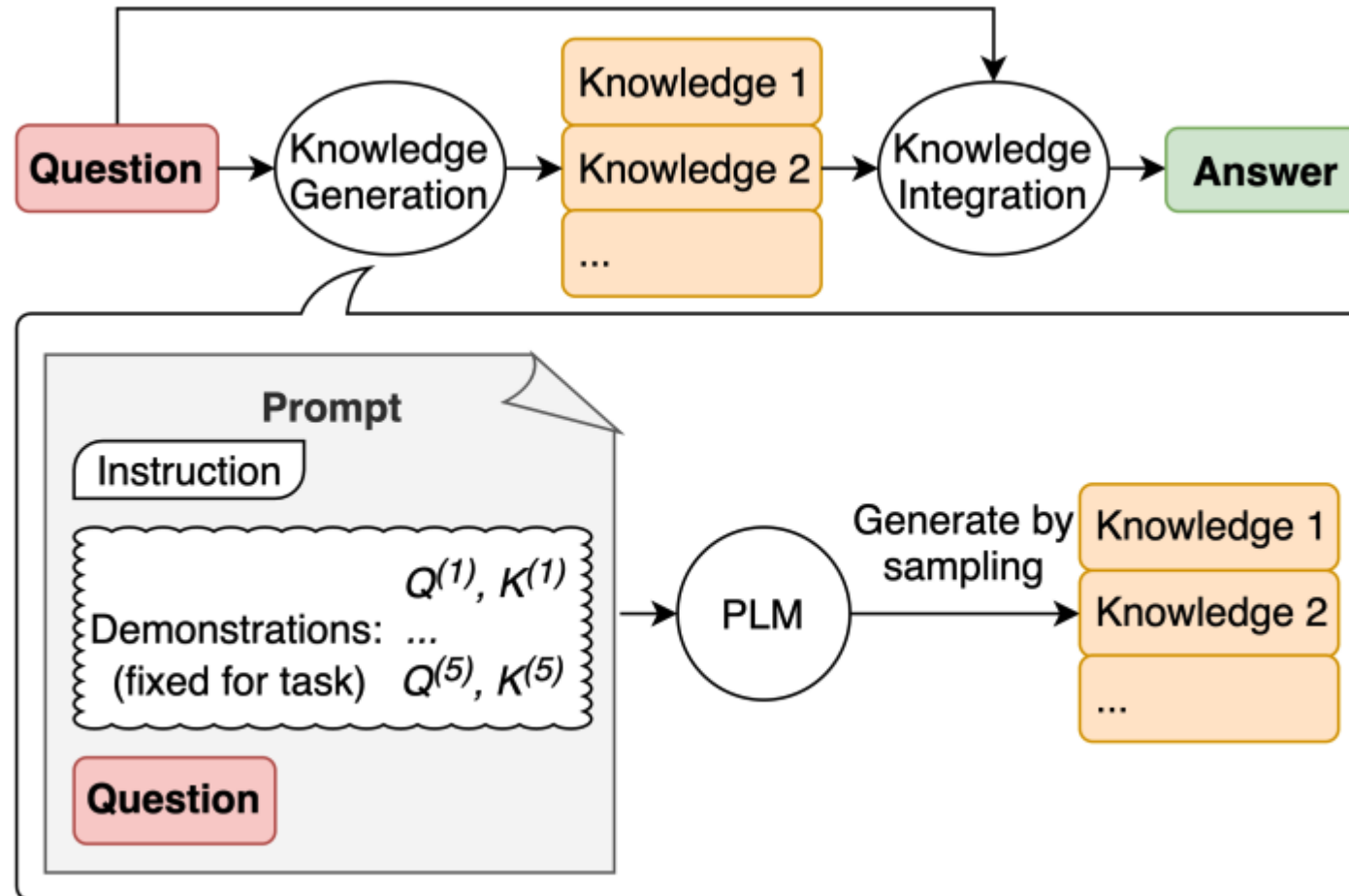
| Aspect             | Chain-of-Thought (CoT) Prompting                                             | Self-Consistency                                                                   |
|--------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Definition         | A prompt engineering method that guides LLMs to solve problems step-by-step. | An advanced method that builds on CoT by generating multiple reasoning paths.      |
| Goal               | Improve reasoning by decomposing complex problems.                           | Enhance reliability by aggregating diverse reasoning paths for consistency.        |
| How it Works       | Prompts the model to think through the solution in a structured way.         | Generates multiple CoT outputs and selects the most frequent or consistent answer. |
| Focus              | Step-by-step logical thinking.                                               | Diversity in reasoning and robustness in final answer.                             |
| Output             | A single reasoning path leading to a solution.                               | Multiple reasoning paths; final answer chosen through majority or consistency.     |
| Use Case Advantage | Useful for complex problems that require intermediate steps.                 | Best for high-stakes or ambiguous tasks needing reliable conclusions.              |

# Generated Knowledge Prompting



- LLMs continue to be improved and one popular technique includes the ability to incorporate knowledge or information to help the model make more accurate predictions.

# Generated Knowledge Prompting



# Generated Knowledge Prompting



"Part of golf is trying to get a higher point total than others. Yes or No?"

Yes.



# Generated Knowledge Prompting



**Input:** Greece is larger than Mexico.

**Knowledge:** Greece is approximately 131,957 sq km, while Mexico is approximately 1,964,375 sq km, making Mexico 1,389% larger than Greece.

**Input:** Glasses always fog up.

**Knowledge:** Condensation occurs on eyeglass lenses when water vapor from your sweat, breath, and ambient humidity lands on a cold surface, cools, and then changes into tiny drops of liquid, forming a film that you see as fog. Your lenses will be relatively cool compared to your breath, especially when the outside air is cold.

**Input:** A fish is capable of thinking.

**Knowledge:** Fish are more intelligent than they appear. In many areas, such as memory, their cognitive powers match or exceed those of 'higher' vertebrates including non-human primates. Fish's long-term memories help them keep track of complex social relationships.

**Input:** Part of golf is trying to get a higher point total than others.

**Knowledge:**

# Generated Knowledge Prompting



**Question:** Part of golf is trying to get a higher point total than others. Yes or No?

**Knowledge:** The objective of golf is to play a set of holes in the least number of strokes. A round of golf typically consists of 18 holes. Each hole is played once in the round on a standard golf course. Each stroke is counted as one point, and the total number of strokes is used to determine the winner of the game.

**Explain and Answer:**

No, the objective of golf is not to get a higher point total than others. Rather, the objective is to play a set of holes in the least number of strokes. The total number of strokes is used to determine the winner of the game, not the total number of points.



# Generated Knowledge Prompting



**Question:** Part of golf is trying to get a higher point total than others. Yes or No?

**Knowledge:** Golf is a precision club-and-ball sport in which competing players (or golfers) use many types of clubs to hit balls into a series of holes on a course using the fewest number of strokes. The goal is to complete the course with the lowest score, which is calculated by adding up the total number of strokes taken on each hole. The player with the lowest score wins the game.

**Explain and Answer:**

Yes, part of golf is trying to get a higher point total than others. Each player tries to complete the course with the lowest score, which is calculated by adding up the total number of strokes taken on each hole. The player with the lowest score wins the game.



# Prompt Chaining



- To improve the reliability and performance of LLMs, one of the important prompt engineering techniques is to break tasks into its subtasks. Once those subtasks have been identified, the LLM is prompted with a subtask and then its response is used as input to another prompt.

# Prompt Chaining



You are a helpful assistant. Your task is to help answer a question given in a document. The first step is to extract quotes relevant to the question from the document, delimited by #####. Please output the list of quotes using <quotes></quotes>. Respond with "No relevant quotes found!" if no relevant quotes were found.

```
#####  
{{document}}  
#####
```

# Prompt Chaining



<quotes>

- Chain-of-thought (CoT) prompting[27]
- Generated knowledge prompting[37]
- Least-to-most prompting[38]
- Self-consistency decoding[39]
- Complexity-based prompting[41]
- Self-refine[42]
- Tree-of-thought prompting[43]
- Maieutic prompting[45]
- Directional-stimulus prompting[46]
- Textual inversion and embeddings[59]
- Using gradient descent to search for prompts[61][62][63][64]
- Prompt injection[65][66][67]

</quotes>



# Prompt Chaining



Given a set of relevant quotes (delimited by <quotes></quotes>) extracted from a document and the original document (delimited by #####), please compose an answer to the question. Ensure that the answer is accurate, has a friendly tone, and sounds helpful.

#####

{{document}}

#####

<quotes>

- Chain-of-thought (CoT) prompting[27]
- Generated knowledge prompting[37]
- Least-to-most prompting[38]
- Self-consistency decoding[39]
- Complexity-based prompting[41]
- Self-refine[42]
- Tree-of-thought prompting[43]
- Maieutic prompting[45]
- Directional-stimulus prompting[46]
- Textual inversion and embeddings[59]
- Using gradient descent to search for prompts[61][62][63][64]
- Prompt injection[65][66][67]

</quotes>

# Prompt Chaining



The prompting techniques mentioned in the document include:

1. Chain-of-thought (CoT) prompting[27]
2. Generated knowledge prompting[37]
3. Least-to-most prompting[38]
4. Self-consistency decoding[39]
5. Complexity-based prompting[41]
6. Self-refine[42]
7. Tree-of-thought prompting[43]
8. Maieutic prompting[45]
9. Directional-stimulus prompting[46]
10. Textual inversion and embeddings[59]
11. Using gradient descent to search for prompts[61][62][63][64]
12. Prompt injection[65][66][67]

Each of these techniques employs unique strategies to enhance or specify the interactions with large language models to produce the desired outcomes.

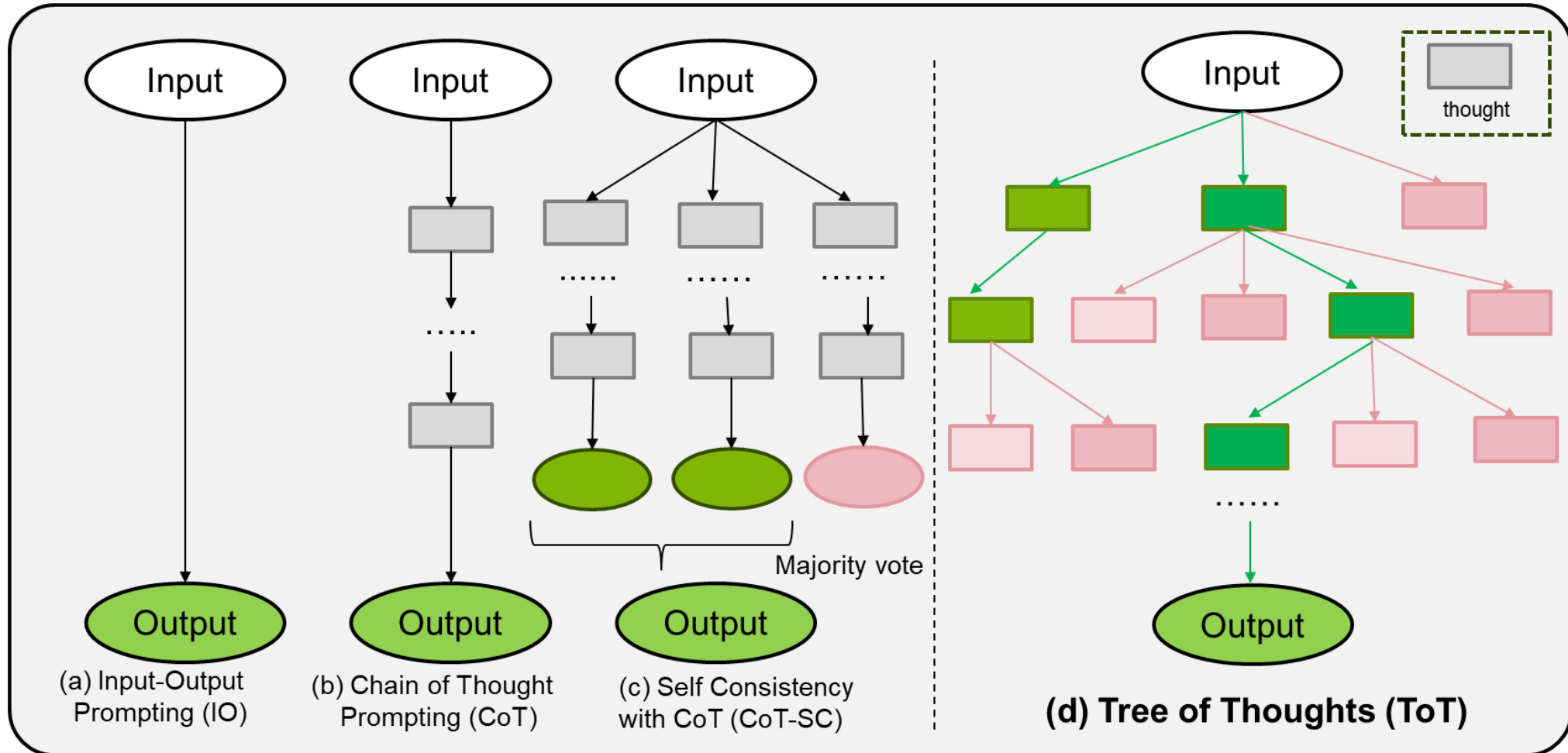




# Tree of Thoughts(ToT)

- The Tree of Thoughts (ToT) technique is an approach that involves building a structured tree-like outline of a conversation to guide an AI model's responses.
- Each node in the tree represents a step in the conversation, and the branching paths help organize the interaction.
- This technique is particularly useful for complex tasks that require multiple layers of guidance.

# Tree of Thoughts(ToT)



# Tree of Thoughts(ToT)



**1. User:** I want to create a text-based Python game.  
**1.1. AI:** What genre or type of game do you have in mind?  
**1.1.1. User:** A simple number guessing game.  
**1.1.1.1. AI:** Great choice! We'll generate a random number and let player guess.  
**1.1.1.1.1. User:** The player should have 3 attempts to guess.  
**1.1.1.1.1.1. AI:** Noted. After 3 incorrect attempts, the game will reveal the correct number.  
**1.1.1.1.2. User:** Also, provide hints if the guess is too high or too low.  
**1.1.1.1.2.1. AI:** Sure! I'll add feedback after each guess to guide the player.  
**1.1.1.1.3. AI:** Any other features or shall we proceed with this design?  
**1.1.1.1.3.1. User:** That sounds good. Let's go with this.  
**1.1.1.1.4. AI:** Perfect! I'll guide you through creating your number guessing game in Python.

# Automatic Reasoning and Tool use (ART)



- The Automatic Reasoning and Tool-use (ART) technique involves instructing an AI model to use reasoning and existing tools to solve complex problems.
- It combines the model's general reasoning abilities with its knowledge of specific tools to achieve the desired outcome.

# Automatic Reasoning and Tool use (ART)



Debug the following Python code that is intended to calculate the factorial of a given positive integer.

**Code Snippet:**

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 2)
```

The provided code snippet attempts to calculate the factorial of a given positive integer. However, it seems to have an issue in the recursive call, where it subtracts 2 from the current n value. It should subtract 1 instead. Here's the corrected code:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```



# Directional Stimulus



- Directional Stimulus Prompting is a technique where you provide a stimulus or input that guides the AI's response in a specific direction.
- This technique can be useful for steering the AI's focus and encouraging it to generate desired outcomes.

# Directional Stimulus



Optimize the following Python code snippet for calculating the sum of even numbers in a list.

**Original Code Snippet:**

```
def sum_even_numbers(numbers):  
    total = 0  
    for num in numbers:  
        if num % 2 == 0:  
            total += num  
    return total
```

**Optimized Code Explanation:**

To optimize the code for calculating the sum of even numbers, you can eliminate the loop and directly calculate the sum using list comprehension. Here's the optimized code:

**Optimized Code Snippet:**

```
def sum_even_numbers(numbers):  
    return sum(num for num in numbers if num % 2 == 0)
```



# ReAct Prompting



- The React Prompting technique involves having a back-and-forth interaction with the AI model, where you provide feedback or react to its responses.
- This technique allows you to guide the AI's output through iterative refinement based on your reactions.

# ReAct Prompting



Generate a Python function that calculates the sum of all even numbers in a list. Please make sure the function is efficient and handles empty lists.

```
def sum_even_numbers(numbers):  
    total = 0  
    for num in numbers:  
        if num % 2 == 0:  
            total += num  
    return total
```



# The “Psychology” of Large Language Models



- They don't “think” then write, they only write
- They don't edit their answer or correct their mistakes
- They don't know what they don't know
- They don't understand their own capabilities
- They don't reflect on their answer
- They predict the next token (word)

Prompt Engineering can [help to make up for those limitations](#) and make the model generate high quality outputs.

# Prompt Engineering Principles for Crafting & Creating Effective Prompts



## Know your model

- Different language models have different capabilities, strengths and weaknesses
- Some models are not trained on code, not trained to perform well with complicated reasoning, or not trained on recent data
- This is the most important principle of crafting effective prompts



## Be detailed and specific

- Provide detailed information about your desired output using direct language
- Focus the prompt on a specific problem, topic, question or output type. Multiple asks in a single prompt can make it difficult for a model to answer effectively



## Provide context

- Contextual information helps the model to solve the task and generate more appropriate responses
- Include conversation history, examples of desired output, or the model's role in the prompt



## Experiment

- Try different approaches to prompting. Crafting effective prompts is more of an art than a science
- Alter prompt wording, change the prompt to align with the model's capabilities, and even ask the model for help to optimize the prompt

# Customizing Copilot Suggestions

# Objectives



- Tailoring Copilot Behavior Through IDE Settings
- Best Practices for Accurate Code Generation

# Enable or Disable Copilot



Copilot 33 Settings Found Backup and Sync Settings

User Workspace

- Text Editor (2)
- Features (3)
  - Terminal (1)
  - Notebook (1)
  - Chat (1)
- Extensions (28)
  - GitHub Copilot (3)
  - GitHub Copilo... (22)
    - Preview (5)
    - Experimental (8)
    - MongoDB (2)
    - Pylance (1)

**Github > Copilot: Enable**  
Enable or disable Copilot completions for specified languages

| Item      | Value |
|-----------|-------|
| *         | true  |
| plaintext | true  |
| markdown  | false |
| scminput  | false |

Add Item

**Github > Copilot > Chat: Locale Override**  
Specify a locale that Copilot should respond in, e.g. `en` or `fr`. By default, Copilot will respond using VS Code's configured display language locale.

auto

**Github > Copilot > Editor: Enable Code Actions**  
✓ Controls if Copilot commands are shown as Code Actions when available

**Github > Copilot > Rename Suggestions: Trigger Automatically**  
✓ Controls whether Copilot generates suggestions for renaming

# AI Pair Programming

# Objectives



- Use Copilot for refactoring, testing, debugging, and adapting code for different frameworks
- Dive into real-world examples for creating APIs, data transformations, and more
- Use Copilot for enabling documentation, security, and also supporting CI/CD integrations

# Best Practices

---

# Objectives



- Tips to make Copilot an effective part of your development workflow
- Understand ethical coding with AI suggestions

# Best Practices



- Open your relevant files
- Provide a top-level comment
- Set Includes and references
- Meaningful names matter
- Provide specific and well-scoped function comments
- Provide sample code
- Remove irrelevant requests
- Navigate through your conversation
- Use the @workspace agent

# Best Practices

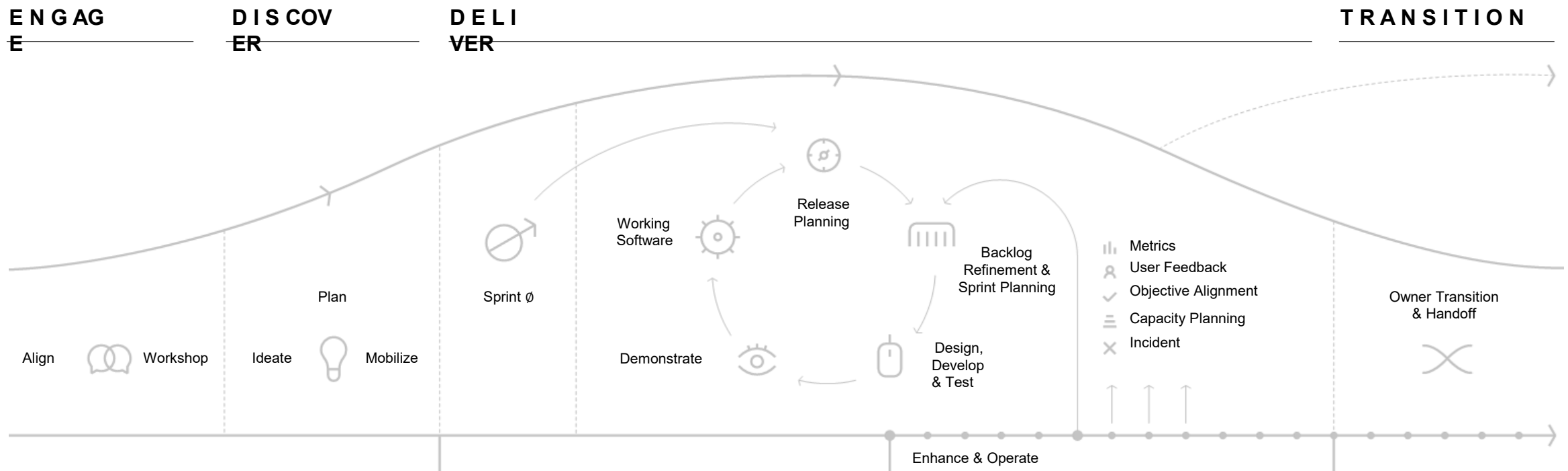


- Highlight relevant code
- Organize your conversations with threads
- Slash Commands for common tasks
- Attach relevant files for reference
- Start with GitHub Copilot Chat for faster debugging

# Gen AI & Software Development



- We are piloting Gen AI at all stages of Product Engineering Methodology to reduce time to market and increase deployment frequency



- **Rapid Prototyping**
- **Backlog Generation**
- **Personas & User Journeys**

- **Code Generation**
- **IaaS CI/CD**
- **Sprint Prioritization**
- **Defect Detection**
- **Team Onboarding**

- **Code Documentation**

**Thank You**